

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2025.DOI

GaussTwin: A Multi-Paradigm Digital Twin Platform with Agentic AI Automation

RISMAN ADNAN¹, SECOND AUTHOR², and THIRD AUTHOR³

¹Affiliation 1, City, Country (e-mail: author1@domain.tld)

²Affiliation 2, City, Country (e-mail: author2@domain.tld)

³Affiliation 3, City, Country (e-mail: author3@domain.tld)

Corresponding author: First Author (e-mail: author1@domain.tld).

ABSTRACT Digital twins increasingly require multi-paradigm simulation to represent complex socio-technical systems that combine continuous dynamics, discrete operational workflows, heterogeneous interacting agents, and learning-enabled decision-making. Yet many existing solutions emphasize a single modeling paradigm, provide limited openness for integrating custom AI/ML, or lack a formal semantics that enables reproducibility and correctness arguments when heterogeneous models are coupled. This paper presents GaussTwin, a web-based digital twin platform built around a mathematically grounded stochastic hybrid semantics that unifies System Dynamics (SD), Discrete-Event Simulation (DES), Agent-Based Modeling (ABM), and Multi-Agent Reinforcement Learning (MARL). GaussTwin is designed for a spreadsheet-first workflow: users author data snapshots and scenario variants in Excel, LLM agents validate and refine these inputs, and the platform synthesizes a typed intermediate representation (GaussIR) that is compiled into executable simulation backends in Rust and Julia. Rust provides safe and high-performance execution with a discrete-event ABM engine (krABMaga) and optional ML/MARL modules (Candle or tch-rs); Julia provides scientific computing and rapid ABM prototyping through Agents.jl. To address reliability risks introduced by LLM-assisted generation, GaussTwin uses a certified agentic compilation pipeline: LLMs propose repairs, model structure, and code, but deterministic validators (schema/type/unit checks, constraint solving, static analysis, and tests) gate acceptance of all artifacts. We present (i) a unified mathematical formulation and embedding theorems for SD/DES/ABM; (ii) deterministic hybrid orchestration algorithms with reproducibility guarantees; (iii) streaming synchronization and state-estimation hooks for real-time data; (iv) a MARL-in-the-loop design; and (v) a deployable microservices architecture, evaluation protocols, and a reproducibility checklist. The result is an open foundation for building auditable, AI-enabled, multi-paradigm digital twins.

INDEX TERMS Digital twin, hybrid simulation, discrete-event simulation, system dynamics, agent-based modeling, multi-agent reinforcement learning, Rust, Julia, LLM agents, agentic compilation, co-simulation.

I. PROBLEM STATEMENT AND CONTRIBUTIONS

A. PROBLEM STATEMENT

We formalize the design of a multi-paradigm, AI-enabled digital twin as a synthesis problem from structured inputs.

Definition 1 (Multi-Paradigm Digital Twin Synthesis Problem). Let $\mathcal{X}$ be a structured dataset (a *snapshot*) encoding entities, parameters, and topology; let $\mathcal{S}$ be a set of scenario specifications (interventions, uncertainties, policies); and let $(t_k, y_k)_{k \geq 1}$ be an observation stream from the physical system. The *multi-paradigm digital twin synthesis problem* is to

construct an executable twin

$$\mathcal{D} = (\mathcal{M}, \mathcal{F}, \mathcal{O}, \mathcal{V})$$

such that:

- 1) (**Expressivity**) $\mathcal{M}$ represents SD, DES, ABM, and MARL components and their couplings.
- 2) (**Semantic correctness**) $\mathcal{M}$ and $\mathcal{O}$ implement a well-defined hybrid semantics; $\mathcal{F}$ yields consistent state/parameter estimates under observations.

- 3) **(Reproducibility)** For a fixed snapshot/scenario and RNG seed, simulation traces are deterministic up to numerical tolerances.
- 4) **(Usability & deployment)** The synthesis pipeline is spreadsheet-first and web-deployable; $\mathcal{V}$ provides interactive visualization and video export.

B. CONTRIBUTIONS

This paper contributes:

- 1) **Unified hybrid semantics.** A generalized hybrid transition semantics that embeds SD, DES, and ABM, and integrates MARL via Markov games.
- 2) **Certified agentic compilation.** A spreadsheet-first LLM-assisted pipeline that synthesizes a typed intermediate representation (GaussIR) and generates Rust/Julia simulation code, gated by deterministic validators.
- 3) **Hybrid orchestration algorithms.** Deterministic time-advance, event processing, streaming synchronization, and MARL training algorithms suitable for reproducible web deployment.
- 4) **Platform blueprint and evaluation plan.** A microservices-oriented architecture, security and governance controls, and evaluation protocols consistent with interoperability roadmaps.

II. INTRODUCTION

DIGITAL TWINS couple a physical system with an executable digital representation that remains synchronized with observations and supports monitoring, prediction, and decision-making. In many application domains, a digital twin must simultaneously model:

- **Continuous dynamics** (e.g., thermodynamics, degradation, energy flows),
- **Discrete operational workflows** (queues, schedules, failures, logistics),
- **Heterogeneous agents** (operators, machines, vehicles, organizations), and
- **Learning-enabled control** (data-driven policies and multi-agent coordination).

This motivates a multi-paradigm approach that combines System Dynamics (SD), Discrete-Event Simulation (DES), Agent-Based Modeling (ABM), and Multi-Agent Reinforcement Learning (MARL).

Despite increasing demand, digital twin engineering remains fragmented. Industrial users frequently begin with spreadsheets containing a snapshot of entities and parameters, plus what-if scenarios. Turning these artifacts into robust simulation models requires domain expertise, software engineering, and repeated validation cycles. LLMs can assist by validating data and generating code, but introduce reliability and governance risks. A production-grade platform therefore needs (i) formal semantics to prevent paradigm coupling errors, (ii) reproducibility guarantees for auditing and debugging, and (iii) guardrails to safely incorporate LLM assistance.

A. GAUSSTWIN AT A GLANCE

GaussTwin is a web platform designed around:

- 1) a **unified stochastic hybrid semantics** for SD, DES, ABM or MARL;
- 2) a **typed intermediate representation** (GaussIR) as a contract between user inputs, LLM agents, validators, and backends;
- 3) **certified agentic compilation:** LLMs propose artifacts, deterministic validators certify them;
- 4) **Rust+Julia execution:** Rust for safe high-performance simulation and ML; Julia for scientific computing and ABM prototyping;
- 5) **web-first visualization** and optional offline video rendering.

B. TECHNOLOGY CHOICES

GaussTwin combines:

- **Rust:** krABMaga is a discrete-event engine for ABM simulation [1]; Candle provides a Rust ML framework with CPU/GPU/WASM backends [5], [6]; tch-rs provides libtorch bindings [7].
- **Julia:** Agents.jl provides a performant ABM framework with peer-reviewed documentation [2]–[4].
- **OpenRouter:** a unified API to multiple LLM providers for routing and fallback [8], [9].
- **Visualization:** web dashboards/animations; optional offline rendering via nannou [10].

C. WHY FORMAL SEMANTICS MATTERS

Coupling SD, DES, and ABM without a common semantics can introduce subtle errors:

- **Time inconsistency:** continuous solvers step over discrete events, producing missed transitions or incorrect ordering.
- **Nondeterminism:** parallel event processing without deterministic ordering breaks reproducibility.
- **Semantic drift:** code generation across multiple languages/backends may change meaning without an IR contract.

We therefore ground GaussTwin in a unified hybrid semantics and derive the orchestration design from correctness theorems.

III. RELATED WORK

A. DIGITAL TWINS: DEFINITIONS, STANDARDS, AND INTEROPERABILITY

Digital twin definitions vary. A widely referenced view frames a twin as a virtual representation connected to its physical counterpart via data flows [27]. In manufacturing, ISO 23247 defines a twin as a fit-for-purpose digital representation of an observable manufacturing element synchronized with that element [24]. Interoperability across multiple twins and tools remains challenging; NIST emphasizes standardized interfaces, orchestration algorithms, governance, and lifecycle maintainability for ecosystems of digital twins [23].

Cloud platforms such as Azure Digital Twins emphasize graph-based representations and modeling languages like DTDL [11]–[13]. Such platforms complement, but do not replace, simulation-first twins when multi-paradigm dynamics and learning-enabled decision making are required.

B. MULTI-METHOD AND HYBRID SIMULATION

System Dynamics (SD), Discrete-Event Simulation (DES), and Agent-Based Modeling (ABM) represent three foundational simulation paradigms, each with distinct strengths:

a: System Dynamics.

SD originated as a methodology for modeling feedback systems using continuous-time equations [28]. Strengths: captures aggregate behavior, feedback loops, and policy effects; efficient for strategic planning. Limitations: assumes homogeneity; struggles with discrete operational details and individual heterogeneity.

b: Discrete-Event Simulation.

DES focuses on event-driven state transitions; DEVS provides a formal specification for discrete-event models [29], [30]. Strengths: explicit representation of queues, resource contention, and stochastic service times; natural for operations research. Limitations: typically lacks continuous dynamics and spatial agent interactions.

c: Agent-Based Modeling.

ABM models complex systems as interacting agents; Bonabeau provides a foundational overview [31], and Agents.jl provides a modern Julia implementation [3]. Strengths: captures heterogeneity, local interactions, emergence, and spatial dynamics. Limitations: computationally intensive; calibration and validation can be challenging.

d: Multi-method integration.

Hybrid and multi-method simulation is studied both academically and in industrial tools: Morgan et al. propose design patterns for mixing DES and SD [32]; Borshchev discusses multi-method modeling in AnyLogic [33]; Howick et al. propose conceptual frameworks for hybrid SD simulation [34]. AnyLogic is a prominent commercial tool supporting SD/DES/ABM combinations [14], [15].

Table 1 summarizes paradigm characteristics and typical coupling patterns.

C. INTEROPERABILITY AND CO-SIMULATION

Co-simulation standards provide coupling interfaces. FMI 3.0 specifies APIs and packaging for model exchange and co-simulation [21]. IEEE HLA standardizes distributed simulation interoperability [22]. GaussTwin draws on these concepts but focuses on internal unification through a single semantics and typed IR, with future adapters to FMI/HLA.

TABLE 1. Simulation Paradigm Characteristics and Coupling Needs

Paradigm	Description
<i>System Dynamics (SD)</i>	
Characteristics	Continuous stocks/flows; ODEs
Coupling needs	Time-step mismatch with DES; aggregation/disaggregation for ABM
<i>Discrete Event Simulation (DES)</i>	
Characteristics	Discrete mode; event queue; guards/resets
Coupling needs	Continuous time in SD; agent state in ABM
<i>Agent-Based Modeling (ABM)</i>	
Characteristics	Individual agent state; spatial/network interactions
Coupling needs	Aggregation to SD stocks; synchronization with DES events
<i>Multi-Agent Reinforcement Learning (MARL)</i>	
Characteristics	Policies π_i ; observation/action spaces
Coupling needs	Decision points in hybrid transitions; reward shaping

D. MARL AND LEARNING-ENABLED TWINS

MARL is formalized through Markov games [36] and surveyed widely [37]. Digital twins increasingly incorporate ML for state estimation, predictive maintenance, optimization, and control [23]. GaussTwin integrates MARL as a first-class modeling element, enabling policies to be trained, validated, and deployed in the simulation loop.

E. LLM AGENTS FOR DATA AND CODE SYNTHESIS

LLMs have shown strong performance on code generation benchmarks [40], [41]. Agentic prompting methods such as ReAct [38] and iterative Self-Refine [39] enable tool-using workflows and self-improvement. However, correctness and governance concerns remain, motivating GaussTwin's certified pipeline where LLM outputs are treated as proposals subject to deterministic validation.

IV. MATHEMATICAL FOUNDATIONS

This section develops a unifying semantics for SD, DES, and ABM, integrates MARL through Markov games, and states key correctness properties (well-posedness, reproducibility, orchestration consistency).

A. NOTATION

Time is $t \in \mathbb{R}_{\geq 0}$. The hybrid state is

$$z(t) \triangleq (x(t), q(t), s(t)) \in \mathcal{Z} \triangleq \mathbb{R}^{n_x} \times \mathcal{Q} \times \prod_{i=1}^{N(t)} \mathcal{S}_i, \quad (1)$$

where $x(t)$ are continuous variables (SD), $q(t)$ a discrete mode (DES regime), and $s(t)$ the stacked agent state (ABM/MARL).

B. SNAPSHOTS AND SCENARIOS AS MATHEMATICAL OBJECTS

Spreadsheets encode a snapshot and scenario variants. We formalize them to connect practical inputs to the semantics.

TABLE 2. Key Notation

Symbol	Meaning
$x(t) \in \mathbb{R}^{n_x}$	continuous state (SD)
$q(t) \in \mathcal{Q}$	discrete mode (DES regime)
$s_i(t) \in \mathcal{S}_i$	agent i state (ABM/MARL)
$\mathcal{E}$	event type set
$\lambda_e(z, t)$	event intensity / hazard
$R_e(z)$	reset map for event e
ε	numerical tolerance (ODE solver)
ξ	RNG seed

Definition 2 (Snapshot). A snapshot is a finite relational structure

$$\mathcal{X} = (\mathcal{U}, \mathcal{P}, \mathcal{G}),$$

where $\mathcal{U}$ is a finite set of entities (with types), $\mathcal{P}$ is a set of parameters/attributes, and $\mathcal{G}$ is a graph of relationships (topology).

Definition 3 (Scenario). A scenario is a tuple

$$\sigma = (\theta, \mathcal{I}, \rho_0, \xi),$$

where θ are parameter overrides or distributions, $\mathcal{I}$ is an intervention schedule (a set of time-stamped modifications to θ or event rules), ρ_0 is an initial state distribution, and ξ is an RNG seed.

The synthesis problem (Definition 1) requires mapping $(\mathcal{X}, \sigma)$ to an executable model that remains faithful to the intended semantics.

C. GENERALIZED HYBRID TRANSITION PROCESS (GHTP)

Assumption 1 (Regularity). For each $q \in \mathcal{Q}$, the vector field $f_q : \mathbb{R}^{n_x} \times \mathcal{S} \times \mathcal{U} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^{n_x}$ is measurable and locally Lipschitz in (x, s) uniformly in t on compact sets. For each event $e \in \mathcal{E}$, the hazard $\lambda_e : \mathcal{Z} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ is measurable and locally bounded, and the reset $R_e : \mathcal{Z} \rightarrow \mathcal{Z}$ is measurable.

Definition 4 (GHTP). A generalized hybrid transition process is specified by $(\mathcal{Q}, f, \mathcal{E}, \lambda, R)$: between event times (τ_k) , the continuous state evolves as

$$\dot{x}(t) = f_{q(t)}(x(t), s(t), u(t), t), \quad t \in (\tau_k, \tau_{k+1}), \quad (2)$$

and event occurrences are governed by total intensity

$$\Lambda(z, t) = \sum_{e \in \mathcal{E}} \lambda_e(z, t). \quad (3)$$

At an event time τ_{k+1} , an event label e_{k+1} is sampled and the state jumps:

$$z(\tau_{k+1}^+) = R_{e_{k+1}}(z(\tau_{k+1}^-)). \quad (4)$$

This representation covers piecewise-deterministic Markov processes (PDMPs) [35], continuous-time jump processes, and deterministic schedules.

D. EMBEDDING SD, DES, AND ABM

a: SD.

If $\mathcal{E} = \emptyset$, Eq. (2) is an ODE system.

b: DES.

If $f_q \equiv 0$ and all state changes occur via R_e , the process reduces to event-driven transitions. DEVS models can be mapped via event scheduling semantics [29], [30].

c: ABM.

Agent interactions generate events that update subsets of agent state. Time-stepped ABM can be represented by deterministic “tick” events; event-driven ABM arises naturally. Engines like krABMaga implement event-driven ABM efficiently [1].

Lemma 1 (Continuous Flow Preservation). Let $\phi_q^{[s,t]}(x_s)$ denote the flow map of Eq. (2) in mode q from time s to t . Under Assumption 1, for fixed (q, s, u) and any $\delta > 0$, there exists $L > 0$ such that for all $x, x' \in B_R(x_0)$ (a ball of radius R around x_0),

$$\left\| \phi_q^{[s, s+\delta]}(x) - \phi_q^{[s, s+\delta]}(x') \right\| \leq e^{L\delta} \|x - x'\|.$$

Proof. By Assumption 1, f_q is locally Lipschitz with constant L on compact sets. Gronwall’s inequality applied to the integral form of the ODE yields the stated bound. $\square$

Theorem 1 (Semantic Embedding of SD/DES/ABM). Any SD model with locally Lipschitz dynamics, any DES model expressible as a finite set of event transitions, and any ABM model whose agent update rules can be written as measurable reset maps over a finite event set can be embedded into a GHTP. Moreover, the embedding preserves the following properties:

- (i) SD feedback loops are preserved as continuous coupling in f_q ;
- (ii) DES priority rules map to deterministic event selection policies;
- (iii) ABM spatial interactions map to neighborhood-parameterized hazards.

Proof. For SD, choose $\mathcal{E} = \emptyset$ and set f_q to the SD vector field. Feedback loops in SD naturally appear as nonlinear terms in $f_q(x, \cdot)$. For DES, set $f_q \equiv 0$ and define $\mathcal{E}$ to correspond to DES transition types; define R_e as the DES update; hazards λ_e encode deterministic schedules (via queued events) or stochastic intensities. Priority rules are implemented through the deterministic total ordering in Assumption 3. For ABM, represent each agent rule as an event type (e.g., move, interact, spawn); define R_e to update the relevant subset of agent states. Spatial neighborhoods enter through parameterization: $\lambda_e(z, t)$ depends on local agent density or proximity. Measurability and finiteness ensure the resulting dynamics satisfy Definition 4. $\square$

E. INTEGRATING MARL VIA HYBRID MARKOV GAMES

Definition 5 (Hybrid Markov Game). A hybrid Markov game is a tuple

$$\mathcal{G} = (\mathcal{Z}, \{\mathcal{O}_i\}_{i=1}^N, \{\mathcal{A}_i\}_{i=1}^N, \mathcal{T}, \{r_i\}_{i=1}^N, \gamma),$$

where $\mathcal{Z}$ is the hybrid state space, $\mathcal{O}_i$ is agent i 's observation space, $\mathcal{A}_i$ its action space, $\gamma \in (0, 1)$ is a discount factor, rewards r_i map states and actions to real values, and $\mathcal{T}$ is the transition kernel induced by the GHTP when actions parameterize hazards, resets, or continuous controls.

This is consistent with standard Markov game definitions used for MARL [36], [37].

F. WELL-POSEDNESS AND REPRODUCIBILITY

Proposition 1 (Existence and Uniqueness Between Events). *Under Assumption 1, for fixed (q, s, u) the ODE in Eq. (2) has a unique maximal solution between events.*

Proof. Fix mode $q \in \mathcal{Q}$, agent state s , and input u . Assumption 1 guarantees that $f_q(\cdot, s, u, t)$ is locally Lipschitz in x uniformly on compact time intervals. By the Picard-Lindelöf theorem, for any initial condition $x(\tau_k) = x_k$ at event time τ_k , there exists a unique maximal solution $x(t)$ on an interval $[\tau_k, \tau_k + \delta)$ until either the next event occurs or the solution reaches the boundary of the domain. $\square$

Definition 6 (Deterministic Simulation Trace). Given initial state z_0 , scenario parameters $\sigma = (\theta, \mathcal{I}, \rho_0, \xi)$, and solver tolerance ε , a simulation produces a trace

$$\mathcal{T}_{\xi, \varepsilon} = \{(t_k, z(t_k), e_k)\}_{k=0}^K.$$

Assumption 2 (Deterministic Orchestration). The platform uses (i) a deterministic RNG seeded by ξ , (ii) a deterministic total order on simultaneous events, and (iii) deterministic numerical integration given (ε, θ) .

Theorem 2 (Reproducibility). *Under Assumption 2, repeated runs with identical $(z_0, \sigma, \varepsilon)$ produce identical traces $\mathcal{T}_{\xi, \varepsilon}$.*

Proof. Given fixed inputs, the ODE integrator yields identical continuous evolution between event checks; deterministic event selection (including tie-breaking) yields the same event sequence; resets are deterministic functions of state. Induction over event steps proves identical traces. $\square$

G. BACKEND TRACE EQUIVALENCE AND CROSS-LANGUAGE PORTABILITY

One goal of GaussIR is to prevent “semantic drift” across backend implementations. We therefore formalize what it means for two backends (e.g., Rust and Julia) to be equivalent, and state a portability theorem that directly supports regression testing via trace hashing.

Definition 7 (Backend Semantics for GAUSSIR). Fix a certified GAUSSIR program $\mathcal{P}$ and a deterministic orchestrator

specification (Algorithm 4). A backend implementation B induces a trace function

$$\Phi_B : (z_0, \sigma, \varepsilon) \mapsto \mathcal{T}_{\xi, \varepsilon}^{(B)}.$$

We say B is GAUSSIR-faithful if it implements the same (f, λ, R) induced by $\mathcal{P}$ and uses the same deterministic event ordering and RNG algorithm.

Theorem 3 (Cross-Backend Trace Equivalence (Numerical Tolerance)). *Let B_1 and B_2 be GAUSSIR-faithful backends for the same certified program $\mathcal{P}$. Assume both use identical RNG algorithm/seed handling, identical deterministic tie-breaking, and ODE solvers whose local truncation errors are bounded by ε . Then for identical inputs $(z_0, \sigma, \varepsilon)$, the traces $\mathcal{T}_{\xi, \varepsilon}^{(B_1)}$ and $\mathcal{T}_{\xi, \varepsilon}^{(B_2)}$ have identical discrete event labels and event times, and their continuous states satisfy*

$$\sup_{t \in [0, T]} \|x_{B_1}(t) - x_{B_2}(t)\| \leq C \varepsilon$$

for a problem-dependent constant C on any finite horizon T where the solution remains in a compact set.

Proof sketch. Discrete-event equivalence follows from identical hazards/guards, identical RNG, and identical deterministic ordering. Continuous-state deviation follows from standard stability bounds for Lipschitz ODE flows under bounded local error; stitching the bounds over inter-event intervals yields the stated $O(\varepsilon)$ envelope. $\square$

Remark 1 (Practical implication). Theorem 3 justifies backend cross-checks: run the same scenario/seed on Rust and Julia, hash the canonical event trace, and assert equality; additionally compare continuous signals within tolerance. This provides a low-cost regression oracle against semantic drift during compiler/runtime evolution.

H. COMPONENT INTERFACES, COMPOSITION, AND CO-SIMULATION

Large digital twins are rarely written as monolithic models; instead they are composed from subsystems (equipment, resources, fleets, buildings, markets) developed by different teams and sometimes executed by different simulators. To support modularity and interoperability, we formalize hybrid components with explicit input/output interfaces and define a composition operator that preserves the GHTP semantics.

Definition 8 (Hybrid Component). A hybrid component is a tuple

$$\mathcal{C} = (\mathcal{Z}, \mathcal{U}, \mathcal{Y}, f, \mathcal{E}, \lambda, R, h),$$

where $\mathcal{Z}$ is the component state space, $\mathcal{U}$ an input space, $\mathcal{Y}$ an output space, $(f, \mathcal{E}, \lambda, R)$ define a GHTP on $\mathcal{Z}$ with inputs $u \in \mathcal{U}$, and $h : \mathcal{Z} \rightarrow \mathcal{Y}$ is an output map.

Definition 9 (Component Composition). Given components $\mathcal{C}_1$ and $\mathcal{C}_2$, a (directed) coupling is specified by measurable maps

$$\phi_{12} : \mathcal{Y}_1 \rightarrow \mathcal{U}_2, \quad \phi_{21} : \mathcal{Y}_2 \rightarrow \mathcal{U}_1.$$

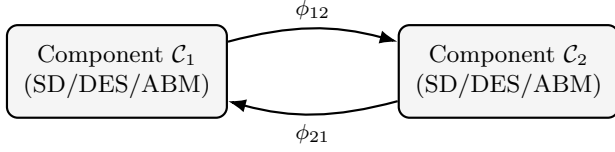


FIGURE 1. Component composition via explicit coupling maps. This abstraction supports modular GAUSSIRcomponents and co-simulation adapters.

The composed system $\mathcal{C}_1 \otimes \mathcal{C}_2$ has product state $z = (z_1, z_2)$, coupled inputs $u_1 = \phi_{21}(h_2(z_2))$, $u_2 = \phi_{12}(h_1(z_1))$, and combined event set $\mathcal{E} = \mathcal{E}_1 \cup \mathcal{E}_2$ with hazards and resets acting on the corresponding subsystem state.

Theorem 4 (Well-Posedness of Composition). *Assume each component satisfies Assumption 1, and the coupling maps ϕ_{12}, ϕ_{21} are locally Lipschitz (for continuous inputs) and measurable (for discrete signals). Then the composed system $\mathcal{C}_1 \otimes \mathcal{C}_2$ defines a valid GHTP on the product state space and is well-posed between events.*

Proof sketch. The coupled vector field on (z_1, z_2) is a composition of locally Lipschitz maps and is therefore locally Lipschitz. Event hazards and resets remain measurable under product construction. Existence and uniqueness between events follow from standard ODE theory; event handling follows from Definition 4. $\square$

Remark 2 (Why this matters for GAUSSTWIN). Theorem 4 motivates GAUSSTWIN's architectural choice to represent subsystem models as GAUSSIRcomponents with explicit I/O mappings, enabling both in-process composition and standards-based federation (FMI/HLA) without semantic ambiguity.

V. GAUSSTWIN PLATFORM OVERVIEW

This section translates the formal objectives into a concrete platform design.

A. END-TO-END WORKFLOW

GaussTwintargets a spreadsheet-first workflow:

- 1) **Authoring:** Users create Excel sheets encoding a snapshot $\mathcal{X}$ and scenarios $\sigma \in \mathcal{S}$.
- 2) **LLM-assisted validation:** An LLM agent proposes repairs, missing values, and consistency checks.
- 3) **Certification:** Deterministic validators enforce schema/type/unit constraints and scenario feasibility.
- 4) **IR synthesis:** The platform produces a typed GaussIRprogram $\mathcal{P}$ that encodes (f, λ, R) and optional MARL hooks.
- 5) **Code generation:** GaussIRcompiles to Rust/Julia backend code and to frontend visualization bindings.

- 6) **Execution and visualization:** Simulations run with deterministic seeds; results are rendered as web animations or exported as video.
- 7) **Real-time sync:** Optional connectors ingest streaming data (OPC UA/MTCConnect) for synchronization and calibration [25], [26].

B. PLATFORM FEATURES AT A GLANCE

Table 3 provides a structured summary of GaussTwin's key features organized by platform layer.

C. WORKFLOW VISUALIZATION

D. KEY DESIGN PRINCIPLE: TREAT LLM OUTPUTS AS UNTRUSTED

LLMs are powerful but probabilistic. GaussTwintherefore treats every LLM-produced artifact (cleaned data, scenarios, IR, code) as untrusted until validated. This principle motivates the certified compilation pipeline in Section VI and is aligned with interoperability roadmaps emphasizing verification, governance, and maintainability [23].

VI. GAUSSIR AND CERTIFIED AGENTIC COMPILATION

A. GAUSSIR DESIGN GOALS

GaussIRacts as a contract between user inputs, LLM agents, validators, and backends. Design goals:

- **Expressive:** SD flows, DES events, ABM agent schemas, MARL hooks.
- **Typed:** static types and optional units for parameters/variables.
- **Composable:** modular components and scenario overlays.
- **Portable:** compile to Rust and Julia with minimal semantic drift.

B. IR CORE SCHEMA

A GaussIRprogram $\mathcal{P}$ consists of:

- **Entities:** typed records with identifiers and relationships (derived from snapshot $\mathcal{X}$).
- **State:** continuous variables x , discrete mode q , and agent state schema s_i .
- **Flows:** equation blocks implementing f_q in Eq. (2).
- **Events:** event types with hazards/guards and resets implementing R_e in Eq. (4).
- **Agents:** update hooks (event-driven or tick-driven) and neighborhood functions.
- **RL:** observation extractors Ω_i , action spaces, reward functions r_i .
- **Scenarios:** parameter distributions and intervention schedules.

C. DETERMINISTIC VALIDATORS

Validators certify $\mathcal{P}$ by enforcing:

- 1) schema/structural conformance (required fields),
- 2) type correctness (domains, enum values),
- 3) unit consistency (dimensional analysis),
- 4) scenario feasibility (constraint solving),

TABLE 3. GAUSSLTWINPlatform Features by Layer

Layer	Features	Implementation Notes
Data & authoring	Excel snapshot templates; schema-aware parsing; unit annotations; relationship graphs	Structured sheets (Entities, Relations, Parameters, Initial-State, Scenarios); JSON intermediate format
Agentic compilation	LLM-assisted data repair; scenario synthesis; iterative refinement; certified IR generation	Algorithm 1; OpenRouter for model routing; deterministic validators (schema, type, unit, constraint, static analysis)
Typed IR (GAUSSIR)	Entity schemas; SD flows / DES events / ABM agents; MARL hooks; scenario overlays	JSON/Protobuf encoding; versioned schema; content-addressable artifact storage
Simulation kernel	Deterministic hybrid orchestration; event queue & ODE integration; reproducible RNG; trace hashing	Algorithm 4; Rust (krABMaga) and Julia (Agents.jl) backends; trace equivalence checks (Theorem 3)
Streaming & sync	Hard overwrite; soft filtering (EKF/UKF/PF); watermarking & replay; industrial connectors	Algorithm 5; OPC UA / MTConnect adapters; replayable observation logs
AI/ML & MARL	In-loop MARL training; policy deployment (advisory/bounded); surrogate models; anomaly detection	Algorithm 6; Candle (Rust ML) or tch-rs (libtorch); Julia ML ecosystem integration
Visualization & re-reporting	Real-time dashboards; scenario comparison; agent animations; report export (PDF/HTML)	Web (WebSocket/SSE); optional offline rendering (nannou); deterministic frame generation (Algorithm 8)
Governance & ops	Artifact versioning & immutability; RBAC & audit logging; reproducibility checklist; experiment sweeps & caching	Run key $\mathcal{R}$; content-addressable registry; cache-aware sweeps (Algorithm 7); Appendix D

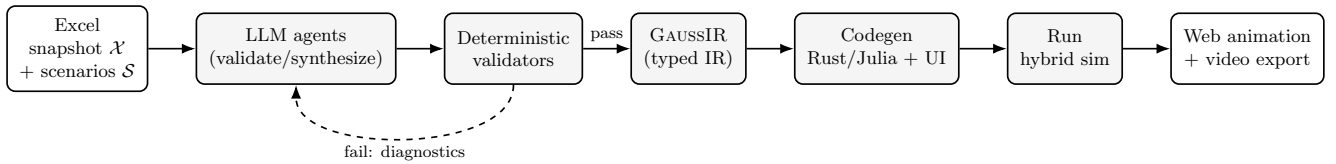


FIGURE 2. End-to-end GAUSSLTWIN workflow from spreadsheets to certified IR, code generation, simulation, and visualization.

- 5) static analysis (unreachable events, cyclic dependencies),
- 6) generated test suites (property tests for invariants).

D. CERTIFICATION GUARANTEES AS A REFINEMENT CONTRACT

From an engineering perspective, GaussIR certification is valuable only if it establishes a clear contract between (i) user intent expressed in snapshots/scenarios, (ii) LLM-proposed artifacts, and (iii) backend execution. We formalize this as a refinement-style guarantee: certification ensures the generated program is within an explicitly defined safe subset and satisfies platform-level invariants required for auditability.

Definition 10 (Certified Subset and Safety Obligations). Let $\mathfrak{S}$ denote a *certified subset* of GAUSSIR programs characterized by syntactic and semantic obligations: (i) Lipschitz continuous flows, (ii) measurable hazards/guards and deterministic resets, (iii) total ordering for simultaneous events, (iv) declared units/types for all externally exposed parameters, and (v) a set of invariants $\mathcal{I}$ (optional) to be enforced by generated tests. We write $\mathcal{P} \in \mathfrak{S}$ if $\mathcal{P}$ satisfies these obligations.

Theorem 5 (Certification Soundness (Platform Contract)). *If*

validators accept a program $\mathcal{P}$ produced by certified agentic compilation (Algorithm 1), then $\mathcal{P} \in \mathfrak{S}$ and:

- 1) (**Well-posedness**) *the induced hybrid dynamics are well-posed between events (Proposition and Assumption 1);*
- 2) (**Reproducibility**) *the deterministic orchestrator yields reproducible traces (Theorem 2);*
- 3) (**Regression oracle**) *invariant-driven tests are sound with respect to declared invariants (Algorithm 3);*
- 4) (**Portability**) *certified backends can be regression-tested for semantic drift via trace equivalence checks (Theorem 3).*

Proof sketch. Each validator enforces a specific obligation of $\mathfrak{S}$ (types/units, constraint feasibility, static analysis preconditions, and test synthesis). The platform contract then follows by composing these obligations with the mathematical results of Section IV and the orchestrator correctness theorem (Theorem 7). $\square$

E. LLM AGENT ROLES

Table 4 summarizes the specialized LLM agents in the certified compilation pipeline, their primary responsibilities, and the deterministic validators that gate their outputs.

TABLE 4. LLM Agent Roles and Validation Gates

Agent	Responsibilities	Validation Gates
Data Validation Agent	Repair missing/invalid values; enforce schema and unit hints; propose normalization and outlier detection	Schema validator; type/unit checker; statistical feasibility tests
Scenario Synthesis Agent	Convert natural language scenario descriptions into structured DSL/GAUSSIR overlays; propose interventions, parameter sweeps, and distribution specs	Constraint solver (feasibility); static analysis (reachability)
Backend Codegen Agent	Generate Rust/Julia modules from GAUSSIR templates; emit tests and invariants; respect template slots	Type checker; invariant-driven tests (Algorithm 3); compilation success
Frontend Codegen Agent	Generate UI bindings and visualization code consistent with GAUSSIR identifiers; produce render maps ψ	Identifier cross-check; lint/format check
Verifier Agent	Run tool-based checks (SMT solvers, linters, test harnesses) and iterative refinement before artifacts are accepted	Meta-validator: checks that all domain validators passed

Algorithm 1 Certified Agentic Compilation to GAUSSIR

Require: Excel files $\mathcal{X}$, schema Σ , unit rules $\mathcal{U}$, constraints $\mathcal{C}$, LLM router $\mathcal{R}$, max iterations K

Ensure: Certified GAUSSIR program $\mathcal{P}$ or failure report

```

1:  $D \leftarrow \text{PARSEXCEL}(\mathcal{X})$ 
2:  $S \leftarrow \text{EXTRACTSTRUCTURE}(D, \Sigma)$ 
3: for  $k = 1$  to  $K$  do
4:    $m \leftarrow \mathcal{R}(\text{task} = \text{validate\_and\_synthesize}, \text{context} = S)$ 
5:    $(\hat{D}, \hat{\mathcal{P}}, \Delta) \leftarrow \text{LLMPROPOSE}(m, D, S)$ 
6:    $v_1 \leftarrow \text{SCHEMAVALIDATE}(\hat{D}, \Sigma)$ 
7:    $v_2 \leftarrow \text{TYPEUNITCHECK}(\hat{\mathcal{P}}, \mathcal{U})$ 
8:    $v_3 \leftarrow \text{CONSTRAINTSOLVE}(\hat{\mathcal{P}}, \mathcal{C})$ 
9:    $v_4 \leftarrow \text{STATICANALYZE}(\hat{\mathcal{P}})$ 
10:  if  $v_1 \wedge v_2 \wedge v_3 \wedge v_4$  then
11:    return  $\hat{\mathcal{P}}$   $\triangleright$  Certified IR
12:  else
13:     $S \leftarrow \text{AUGMENTCONTEXT}(S, v_1, v_2, v_3, v_4)$   $\triangleright$  feedback
14:  end if
15: end for
16: return Failure report with diagnostics

```

a: Agent orchestration and feedback loops.

Agents operate in a refinement loop (Algorithm 1): when validators reject an artifact, diagnostics are fed back to the proposing agent as structured context, enabling targeted repairs. This iterative process typically converges within 3–5 iterations for well-formed inputs; malformed inputs or constraint violations may require human intervention.

F. CERTIFIED AGENTIC COMPILATION ALGORITHM

Algorithm 1 formalizes certified compilation from spreadsheets to GaussIR.

Algorithm 2 Utility-Aware LLM Routing

Require: candidate models $\mathcal{M}$, task τ , weights (α, β, γ) , logs $\mathcal{L}$

```

1: estimate  $\hat{q}(m)$  success probability from  $\mathcal{L}$ 
2: obtain cost  $c(m)$  and latency  $\ell(m)$  metadata
3: select  $m^* \leftarrow \arg \max_{m \in \mathcal{M}} \alpha \hat{q}(m) - \beta c(m) - \gamma \ell(m)$ 
4: return  $m^*$ 

```

G. LLM ROUTING VIA OPENROUTER

OpenRouter provides a unified API and supports routing across providers [8], [9]. We propose a utility-aware selection strategy for routing models by task criticality (validation vs codegen) and empirical success rates.

H. TEMPLATE-RESTRICTED CODE GENERATION AND TEST SYNTHESIS

To reduce semantic drift and security risk, GaussTwin avoids free-form code generation whenever possible. Instead, it uses template-restricted generation:

- Backends expose a small library of verified primitives (event queue, RNG, ODE integrator, agent container).
- Code generation fills typed “slots” (equation coefficients, guard expressions, reset assignments) rather than emitting arbitrary control flow.
- LLMs are only allowed to propose content for these slots and must reference existing GaussIR identifiers; validators reject unknown symbols.

A key benefit is that the majority of the execution logic remains within audited runtime libraries, while generated code remains small and testable.

a: Property tests from invariants.

GaussIR supports optional invariant annotations (e.g., non-negative inventories, bounded capacities, conservation laws).

Algorithm 3 Invariant-Driven Test Synthesis

Require: GAUSSIRprogram $\mathcal{P}$ with invariant set $\mathcal{I}$ and event set $\mathcal{E}$

```

1: Emit test harness with deterministic RNG seed  $\xi_{\text{test}}$ 
2: for all  $e \in \mathcal{E}$  do
3:   for all  $I \in \mathcal{I}$  do
4:     generate random state  $z$  satisfying declared domains
5:     compute  $z' \leftarrow R_e(z)$ 
6:     assert  $I(z') = \text{true}$  (or bounded violation)
7:   end for
8: end for

```

From these, the compiler emits property-based tests that sample random states and check invariants across resets.

Remark 3. Invariant-driven tests do not prove correctness, but they provide a strong guardrail against common LLM-induced defects (wrong field names, sign errors, missing bounds) and support regression testing when scenarios evolve.

I. SOUNDNESS OBJECTIVE

Definition 11 (Well-Typed IR). We write $\mathcal{P} \vdash \text{ok}$ if $\mathcal{P}$ passes schema/type/unit/constraint checks.

Theorem 6 (Compilation Soundness (Core Subset)). *Consider a core subset of GAUSSIR where continuous flows are Lipschitz ODEs, events have measurable guards and deterministic resets, and agent updates occur only through events. If $\mathcal{P} \vdash \text{ok}$ and code generation emits backend code $C = \text{COMPILE}(\mathcal{P})$ using the deterministic orchestrator in Section VII, then executing C simulates the GHTP semantics induced by $\mathcal{P}$ up to numerical tolerance ε .*

Proof sketch. Well-typedness guarantees f_q , guards, and resets are defined over declared domains. The orchestrator implements a deterministic conservative time-advance scheme; convergence to the intended hybrid dynamics follows from ODE solver convergence and deterministic event handling (Theorem 7). Thus the generated code preserves IR semantics within numerical error. $\square$

VII. HYBRID SIMULATION KERNEL

This section designs the simulation kernel executing a GHTP and supporting SD–DES–ABM couplings and MARL decision points.

A. KERNEL REQUIREMENTS

- **Determinism and reproducibility** (Theorem 2).
- **Efficient event handling** for DES/ABM.
- **Stable continuous integration** for SD.
- **Composable coupling** across paradigms and backends.

B. COUPLING PATTERNS

Table 5 summarizes common coupling patterns.

TABLE 5. Typical Multi-Paradigm Coupling Patterns

Pattern	Description
SD $\rightarrow$ DES	Continuous variables parameterize event intensities or service times
DES $\rightarrow$ SD	Event history updates SD inputs (e.g., utilization drives energy ODEs)
ABM $\leftrightarrow$ DES	Agent actions schedule events; events update agent state and queues
MARL $\rightarrow$ ABM/DES	Policies choose actions that parameterize hazards/resets (dispatch, routing)
Streaming $\rightarrow$ all	Observations update state/parameters via over-write or filtering

Algorithm 4 Deterministic Hybrid Orchestration (DHO)

Require: z_0 , end time T , max step $h_{\max}$, RNG seed ξ , tolerance ε

```

1: init RNG( $\xi$ );  $t \leftarrow 0$ ;  $z \leftarrow z_0$ ; init event queue  $\mathcal{Q}_e$ 
2: while  $t < T$  do
3:    $t_{\text{next}} \leftarrow \min(t + h_{\max}, T)$ 
4:    $\tau \leftarrow \text{NEXTEVENTTIME}(\mathcal{Q}_e)$ 
5:    $t_{\text{step}} \leftarrow \min(t_{\text{next}}, \tau)$ 
6:    $x \leftarrow \text{INTEGRATEODE}(x, f_q, [t, t_{\text{step}}], \varepsilon)$ 
7:    $t \leftarrow t_{\text{step}}$ 
8:    $\mathcal{E}_{\text{en}} \leftarrow \text{ENABLEDEVENTS}(z, t)$ 
9:   if  $\mathcal{E}_{\text{en}} \neq \emptyset$  then
10:     $e^* \leftarrow \text{SELECTEVENTDET}(\mathcal{E}_{\text{en}})$ 
11:     $z \leftarrow R_{e^*}(z)$ 
12:   end if
13:    $\mathcal{Q}_e \leftarrow \text{RESCHEDULE}(\mathcal{Q}_e, z, t)$ 
14:   Record  $(t, z, e^*)$ 
15: end while

```

C. DETERMINISTIC HYBRID ORCHESTRATION

We propose deterministic hybrid orchestration (Algorithm 4), compatible with conservative co-simulation and FMI-style stepping [21].

Assumption 3 (Event Separability or Total Ordering). Either (i) almost surely no two enabled events share the same event time, or (ii) the platform defines a deterministic total order on simultaneous events.

Theorem 7 (Consistency of DHO with GHTP). *Under Assumptions 1, 2, and 3, Algorithm 4 produces an event sequence consistent with the GHTP semantics and continuous trajectories converging to the GHTP flow as $h_{\max} \rightarrow 0$ and $\varepsilon \rightarrow 0$.*

Proof sketch. Between events, convergence follows from numerical analysis for Lipschitz ODEs. Deterministic ordering resolves simultaneous events. As $h_{\max} \rightarrow 0$, event detection error vanishes and the resulting trace converges to the GHTP semantics. $\square$

Proposition 2 (Event Detection Error Bound). *Assume hazards $\lambda_e(z, t)$ are Lipschitz continuous in t with constant L_λ .*

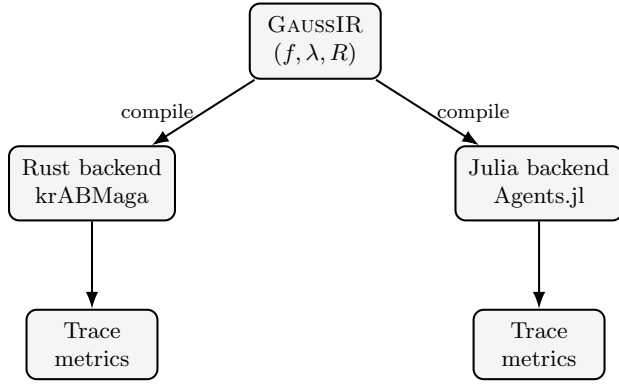


FIGURE 3. Backends compile the same typed semantics from GAUSSIR.

If Algorithm 4 uses step size $h_{\max}$, the probability of missing an event in a time interval $[t, t + h_{\max}]$ is bounded by:

$$P(\text{miss event}) \leq \exp\left(-\int_t^{t+h_{\max}} \lambda_{\min}(\tau) d\tau\right) \cdot (1 - \exp(-L_{\lambda} h_{\max})).$$

where $\lambda_{\min}(t) = \min_e \lambda_e(z(t), t)$ over enabled events.

Proof sketch. Event detection failure occurs when an event fires within $[t, t + h_{\max}]$ but the guard is not checked. For small $h_{\max}$, the firing probability is approximately $\int_t^{t+h_{\max}} \lambda(s) ds \approx \lambda(t)h_{\max}$. The Lipschitz bound on λ controls how quickly the intensity changes, yielding the stated error envelope. In practice, conservative time-stepping with $h_{\max} \ll 1/\lambda_{\max}$ ensures negligible miss rates. $\square$

D. BACKEND EXECUTION (RUST AND JULIA)

The same GaussIR semantics can compile to:

- **Rust backend:** krABMaga for event-driven ABM [1]; optional ML/MARL via Candle/tch-rs [5], [7].
- **Julia backend:** Agents.jl for ABM and scientific computing [3].

E. COMPLEXITY AND SCALABILITY CONSIDERATIONS

Let N be number of agents, M number of events processed, and n_x number of continuous variables. A typical execution cost decomposes as:

$$\text{Cost} \approx \underbrace{O(M \log M)}_{\text{event queue}} + \underbrace{O(n_x \cdot N_{\text{steps}})}_{\text{ODE integration}} + \underbrace{O(M \cdot c_R)}_{\text{resets}},$$

where c_R is the average cost of applying a reset (often local to a few agents). Rust implementations can exploit deterministic parallelism for independent computations (e.g., agent perception) while preserving deterministic ordering for state updates.

Algorithm 5 Streaming Synchronization Loop

Require: Orchestrator $\mathcal{O}$, filter $\mathcal{F}$, stream connector $\mathcal{C}$

```

1: init belief  $\hat{z}_0$ ;  $t_0$ 
2: while service active do
3:    $(t_k, y_k) \leftarrow \mathcal{C}.\text{NEXTOBSERVATION}()$ 
4:    $\hat{z}_k^- \leftarrow \mathcal{O}.\text{ADVANCE}(\hat{z}_{k-1}^+, t_{k-1}, t_k)$ 
5:    $\hat{z}_k^+ \leftarrow \mathcal{F}(\hat{z}_k^-, y_k)$ 
6:   emit updated state + uncertainty
7: end while
  
```

VIII. STREAMING SYNCHRONIZATION AND STATE ESTIMATION

Digital twins often ingest real-time data through industrial standards such as OPC UA and MTConnect [25], [26]. GaussTwin supports hard and soft synchronization.

A. HARD SYNCHRONIZATION

At observation times t_k , overwrite selected state variables:

$$\hat{z}_k^+ = \text{Overwrite}(\hat{z}_k^-, y_k).$$

This is appropriate for directly observed states and low latency.

B. SOFT SYNCHRONIZATION VIA FILTERING

For latent variables and noisy observations, apply an update:

$$\hat{z}_k^+ = \mathcal{F}(\hat{z}_k^-, y_k),$$

where $\mathcal{F}$ may implement EKF/UKF or particle filtering [42], [43].

C. LATENCY, WATERMARKING, AND REPLAY

To ensure deterministic replay, the platform should record:

- observation timestamps and payload hashes,
- ingestion latency and watermark state,
- filter configuration and random seeds.

This enables auditability and deterministic re-simulation of streaming experiments.

IX. AI/ML AND MARL INTEGRATION

A. ML HOOKS IN THE TWIN LIFECYCLE

GaussTwin supports ML integration for:

- **State estimation:** learned observation/transition models.
 - **Surrogates:** replace expensive simulation components.
 - **Anomaly detection:** monitor deviations between physical and simulated states.
 - **Optimization:** search over scenario parameters or policies.
- Rust ML can use Candle [5] (CPU/GPU/WASM) or tch-rs [7]; Julia can use scientific ML libraries.

B. MARL TRAINING IN THE LOOP

MARL policies interact with the simulation at decision points.

Proposition 3 (Hybrid Step Measurability for Policy Gradients). Assume the reset maps $\{R_e\}$ are differentiable almost

Algorithm 6 MARL Training Loop Inside GAUSTRWIN

Require: hybrid game $\mathcal{G}$, policies $\{\pi_i\}$, optimizer $\mathcal{O}_{\text{opt}}$, episodes M

```

1: for  $m = 1$  to  $M$  do
2:   init  $z_0; z \leftarrow z_0$ 
3:   while not terminal do
4:     for all agents  $i$  do
5:        $o_i \leftarrow \Omega_i(z); \text{sample } a_i \sim \pi_i(\cdot | o_i)$ 
6:     end for
7:     execute hybrid step  $z' \sim \mathcal{T}(\cdot | z, a)$ 
8:     store transition  $(z, o, a, r, z'); z \leftarrow z'$ 
9:   end while
10:  update  $\{\pi_i\}$  using stored transitions
11: end for

```

everywhere with respect to continuous action parameters, and hazards $\{\lambda_e\}$ are differentiable in actions. Then the expected return $J(\pi) = \mathbb{E}_{\tau \sim \pi} [\sum_t \gamma^t r_t]$ is differentiable with respect to policy parameters θ , and unbiased gradient estimates can be constructed via:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | o_t) \Phi_t \right],$$

where Φ_t is a return or advantage estimator.

Proof sketch. The likelihood ratio (REINFORCE) trick applies because the stochastic hybrid process induces a trajectory distribution that is differentiable in policy parameters through the action sampling distribution, even when discrete events are present. Differentiability of hazards and resets with respect to continuous actions ensures that the score function $\nabla_{\theta} \log \pi_{\theta}$ can be pushed through the expectation. $\square$

Lemma 2 (Convergence Rate of Multi-Agent Policy Gradient (Cooperative Case)). *In the cooperative setting where all agents share a common reward $r(z, a) = r_1 = \dots = r_N$, and under Lipschitz continuity and bounded gradients, a centralized training / decentralized execution policy gradient algorithm with step size $\alpha_k = O(1/k)$ converges to a stationary point at rate $O(1/\sqrt{K})$ in expectation over K episodes.*

Proof sketch. Standard non-convex stochastic optimization analysis: bounded variance of gradient estimates and Lipschitz objective yield $O(1/\sqrt{K})$ rate to an ε -stationary point. Cooperation simplifies the game to a single-objective MDP from the centralized perspective. $\square$

C. SAFE POLICY DEPLOYMENT

For safety-critical settings, GaussTwin supports:

- advisory mode (recommendations only),
- bounded-control mode (actions projected onto safe sets),
- fallback policies when anomalies are detected.

In constrained settings, the twin can integrate algorithms for constrained Markov decision processes, such as Constrained Policy Optimization (CPO), to learn policies while

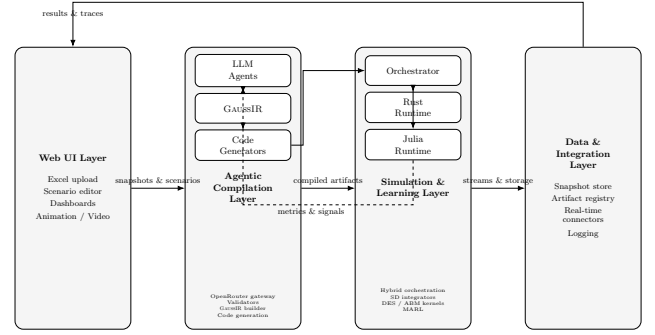


FIGURE 4. Layered GAUSTRWIN architecture integrating web UI, agentic compilation, hybrid simulation, and real-time data connectors.

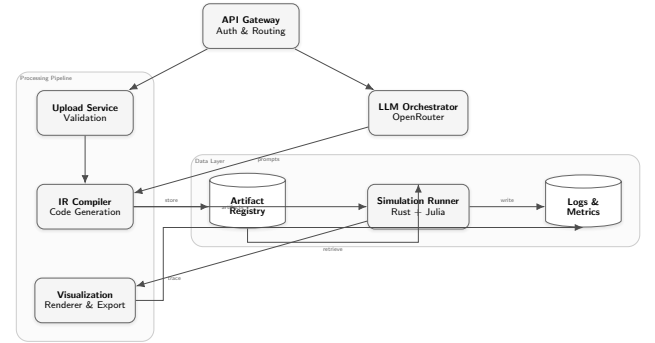


FIGURE 5. Microservices deployment architecture for GAUSTRWIN showing service separation and scaling patterns.

explicitly controlling expected constraint violations [44]. Even when such methods are used, GaussTwin recommends runtime safety filters as a final line of defense.

X. SYSTEM ARCHITECTURE AND IMPLEMENTATION BLUEPRINT**A. LAYERED ARCHITECTURE**

Figure 4 shows the layered architecture.

B. MICROSERVICES DEPLOYMENT OPTION

A practical deployment can follow a microservices pattern to decouple concerns (upload/validation, compilation, simulation, visualization) and scale compute-intensive tasks independently.

a: Determinism under distribution.

Distributed execution introduces potential nondeterminism (e.g., due to parallel scheduling). GaussTwin preserves Theorem 2 by ensuring that all randomness and event ordering are decided inside a deterministic orchestrator; microservices only execute pre-determined steps and report results.

C. CONTROL PLANE VS. DATA PLANE

To make the platform deployable in regulated environments, we separate responsibilities into:

- **Control plane:** authoring, certification, versioning, experiment definitions, access control, and audit logging.
- **Data plane:** simulation execution, streaming synchronization, MARL training/inference, and trace/metric emission.

This separation clarifies security boundaries: user inputs and LLM outputs are handled in the control plane and become immutable, versioned artifacts before any execution in the data plane.

D. DATA FLOW ARCHITECTURE AND MESSAGE CONTRACTS

To make the platform concrete and implementable, we specify the key data flows and message schemas exchanged between services.

a: Upload-to-IR pipeline.

- 1) User uploads Excel file(s) (binary format: .xlsx, typically 100KB–10MB).
- 2) Ingestion service parses to JSON structure (schema-aware); emits snapshot candidate $\hat{\mathcal{X}}$ and validation diagnostics.
- 3) Agentic synthesis service receives $\hat{\mathcal{X}}$ + schema Σ ; LLM proposes repairs Δ (JSON patch format).
- 4) Certification service applies Δ and runs validators (Section VI); outputs: certified GaussIR $\mathcal{P}$ (JSON/Protocol Buffers, typically 10KB–1MB), signed certificate hash.
- 5) Artifact registry stores ($\mathcal{X}$, $\mathcal{P}$, hashes) and returns immutable artifact ID.

b: Simulation execution pipeline.

- 1) User selects scenario σ and seeds $\{\xi_r\}$; frontend constructs run request $\mathcal{R} = (\text{ArtifactID}, \sigma, \xi, \epsilon)$.
- 2) Simulation runner checks cache (artifact registry); if miss, retrieves $\mathcal{P}$ and generates backend code (Rust/Julia).
- 3) Backend executes orchestrator (Algorithm 4); emits trace stream (Protocol Buffers or Arrow format, typical rate: 100–10K events/sec).
- 4) Visualization service consumes trace stream, applies render map ψ (Section XII), and pushes frames to frontend via WebSocket or Server-Sent Events.

c: Streaming synchronization pipeline.

- 1) Streaming gateway subscribes to OPC UA/MTConnect endpoints; receives observations (t_k, y_k) (typical rate: 1–100 Hz).
- 2) Gateway applies watermarking and logging; forwards observation messages to simulation runner.
- 3) Runner invokes filter $\mathcal{F}$ (Algorithm 5); updates belief state $\hat{z}_k^+$ and emits updated state + uncertainty to visualization.

d: Message schema summary.

Table 6 enumerates key message types and their typical payload sizes.

TABLE 6. Key Message Types and Payload Characteristics

Message	Typical size	Encoding
Snapshot JSON	10KB–10MB	JSON
GAUSSIRprogram	10KB–1MB	JSON / Protobuf
Scenario request	1–10KB	JSON
Trace event	100–500 bytes	Protobuf / Arrow
Observation update	50–200 bytes	Protobuf
Visualization frame	10–100KB (raster)	PNG / WebP

E. CORE SERVICES AND RESPONSIBILITIES

Table 7 summarizes a concrete service decomposition aligned with the formal artifacts ($\mathcal{X}$, σ , $\mathcal{P}$, $\mathcal{R}$).

F. CAPABILITY MATRIX (WHAT THE PLATFORM ACTUALLY DOES)

To avoid “architecture-only” descriptions, Table 8 enumerates user-visible capabilities and how they map to the formal model.

G. EXECUTION MODES (BATCH, INTERACTIVE, AND REAL-TIME)

The same certified artifact bundle supports three operational modes:

- 1) **Batch simulation:** scenario sweeps and Monte Carlo runs for decision support.
- 2) **Interactive “what-if”:** low-latency runs with UI-driven interventions and instant visual feedback.
- 3) **Real-time twin:** streaming synchronization plus periodic calibration; optional closed-loop control under bounded policies.

Mode selection impacts resource planning (latency vs throughput) but does not change the semantic contract, which remains anchored in GaussIR and deterministic orchestration.

H. RUST-JULIA INTEGRATION PATTERNS

We propose three integration patterns:

- 1) **Service separation (recommended):** run Rust and Julia backends as separate services communicating via gRPC/HTTP; simplifies deployment and scaling.
- 2) **In-process embedding:** embed Julia in a Rust process via C ABI; lower latency but higher complexity.
- 3) **Standards-based federation:** export components as FMUs or HLA federates [21], [22] for interoperability.

I. TECHNOLOGY MAPPING

J. ARTIFACT VERSIONING AND REPRODUCIBILITY

Each run is defined by a versioned tuple:

$$\mathcal{R} = (\text{SnapshotHash}, \text{ScenarioHash}, \text{IRHash}, \text{CodeHash}, \xi, \epsilon).$$

This supports deterministic replay and regression testing.

K. API AND INTEGRATION PATTERNS

To support programmatic access and integration with external systems, GaussTwin exposes RESTful and gRPC APIs at multiple layers.

TABLE 7. Reference Microservice Decomposition for GAUSSLTWIN

Service	Primary Responsibility	Key Interfaces / Outputs
Project & RBAC	Tenancy; users; roles; policy enforcement	Project IDs; role policies; audit events
Ingestion & Parsing	Excel upload; schema extraction; normalization	Snapshot $\mathcal{X}$; diagnostics
Agentic Synthesis	LLM-driven proposals for repairs/IR overlays	Proposed $\hat{\mathcal{P}}$; change set Δ
Certification	Deterministic validators + proof/test gates	Certified GAUSSIR $\mathcal{P}$; signed report
Artifact Registry	Content-addressable storage of $(\mathcal{X}, \sigma, \mathcal{P}, C)$	Run key $\mathcal{R}$; immutable bundles
Simulation Runner	Deterministic hybrid orchestration + backends	Trace stream; metrics; trace hash
Streaming Gateway	OPC UA/MTConnect connectors; watermarking; replay logs	Observation log; replay tokens
MARL Service	Training/inference hooks bound to decision points	Policy artifacts; evaluation metrics
Visualization & Reporting	Dashboards; scenario comparison; exports	Plots; reports; video frames

TABLE 8. GAUSSLTWINCapability Matrix

Capability	Concrete functionality
Spreadsheet-first modeling	Excel templates for entities/relations/parameters/scenarios with schema enforcement
Certified agentic compilation	LLM proposals gated by schema/type/unit/constraint/static checks (Algorithm 1)
Multi-paradigm execution	SD flows + DES events + ABM agents + MARL decision points (Section VII)
Deterministic replay	Run keys $\mathcal{R}$, trace hashing, deterministic ordering (Theorem 2)
Cross-backend regression	Rust vs. Julia trace equivalence checks (Theorem 3)
Real-time synchronization	Hard overwrite + soft filtering loops (Section VIII)
Scenario sweeps & optimization	Cache-aware sweeps, UQ, black-box optimization (Section XI)
Governance and auditability	Versioned artifacts, signed certification reports, RBAC policies, replayable logs

TABLE 9. Technology Mapping in GAUSSLTWIN

Requirement	Candidate technology
ABM rapid prototyping	Agents.jl (Julia) [2]
High-scale DES/ABM	krABMaga (Rust) [1]
ML/MARL training/inference	Candle [5] or tch-rs [7]
LLM routing	OpenRouter [9]
Real-time industrial connectivity	OPC UA, MTConnect [25], [26]
Visualization/video	Web + nannou [10]

a: Control plane APIs.

- `POST /projects/{id}/snapshots`: upload Excel snapshot; returns parsing diagnostics and snapshot ID.
- `POST /projects/{id}/compile`: trigger certified agentic compilation; returns job ID and streams validation progress.
- `GET /artifacts/{hash}`: retrieve immutable artifact (snapshot, IR, code bundle) by content hash.
- `POST /scenarios`: define scenario overlays and intervention schedules; returns scenario ID.

b: Data plane APIs.

- `POST /simulations/run`: submit run request $\mathcal{R}$; returns trace stream endpoint (WebSocket or gRPC stream).
- `GET /simulations/{runID}/trace`: retrieve completed trace (Arrow/Parquet format).
- `POST /streaming/subscribe`: establish real-time observation stream; accepts OPC UA/MTConnect endpoint config.
- `POST /marl/train`: launch MARL training job; returns policy artifact ID upon convergence.

c: Query and analytics APIs.

- `POST /experiments/sweep`: define scenario sweep; platform schedules parallel runs and returns aggregated metrics.
- `GET /experiments/{sweepID}/results`: retrieve summary statistics (means, quantiles, traces) in JSON or CSV.

d: Standards-based adapters (future).

- **FMI export**: package certified GaussIRcomponents as FMUs (Functional Mock-up Units) for co-simulation with third-party tools (MATLAB/Simulink, Modelica, LabVIEW).
- **HLA federation**: expose GaussTwinsimulations as HLA federates for distributed simulation interoperability.
- **DTDL export**: map GaussIRentity schemas to Azure Digital Twins DTDL for cloud twin graph integration.

L. PERFORMANCE AND SCALABILITY SPECIFICATIONS

Table 10 provides target performance specifications for reference deployment configurations.

a: Scaling strategies.

- 1) **Horizontal scaling**: distribute scenario sweep runs across worker pools (Kubernetes pods, AWS Batch jobs).
- 2) **Vertical scaling**: for single large runs, use multi-core parallelism for independent agent updates; deterministic ordering for state updates.

TABLE 10. Target Performance Specifications (Indicative)

Metric	Target / Notes
Event throughput (Rust backend)	10^5 – 10^6 events/sec (single core; depends on reset complexity)
Agent count (ABM, spatial indexing)	10^4 – 10^6 agents (memory-bound; scales with available RAM)
Continuous state dimension	10^2 – 10^4 ODEs (stiff solvers for large systems)
Trace streaming latency	<100 ms (WebSocket; depends on network)
Observation ingestion rate	100–10K updates/sec (depends on filter complexity)
Scenario sweep parallelism	Auto-scaling workers; limited by cloud quota
Artifact storage	Content-addressable; deduplication; immutable
Deterministic replay variance	< 10^{-6} relative error (floating-point tolerance)

- 3) **Spatial decomposition:** partition large spatial ABM domains into subregions with boundary synchronization (future extension).
- 4) **Surrogate acceleration:** replace expensive simulation sub-models with ML surrogates (trained offline, validated online).

XI. EXPERIMENTATION, SCENARIO SWEEPS, AND OPTIMIZATION

Digital twins are most valuable when they support rapid experimentation: users compare scenarios, quantify uncertainty, and search for interventions or policies that optimize objectives under constraints. GaussTwintherefore treats experiments as first-class objects built on top of versioned artifacts.

A. DETERMINISTIC EXPERIMENT DEFINITION

An experiment is defined by a finite set of scenarios $\{\sigma_j\}_{j=1}^J$, a set of RNG seeds $\{\xi_r\}_{r=1}^R$ (for stochastic models), and a metric map g that extracts scalar or vector outcomes from traces:

$$m_{j,r} = g(\mathcal{T}_{\xi_r, \varepsilon}(\sigma_j)).$$

Aggregates such as means, quantiles, and risk measures are computed across r and compared across j .

B. CACHING AND PARALLEL EXECUTION

Because each simulation run is defined by the artifact tuple $\mathcal{R}$ (Section X), results can be cached and reused across experiments. Algorithm 7 outlines a cache-aware scenario sweep.

C. OPTIMIZATION LOOPS

Many twin objectives are black-box with respect to parameters (especially when DES/ABM and discrete decisions dominate), motivating gradient-free optimization. GaussT-

Algorithm 7 Cache-Aware Scenario Sweep

Require: scenario set $\{\sigma_j\}$, seed set $\{\xi_r\}$, tolerance ε , artifact registry $\mathcal{A}$

```

1: for  $j = 1$  to  $J$  do
2:   for  $r = 1$  to  $R$  do
3:     compute run key  $\mathcal{R}_{j,r} = (\text{hashes}, \xi_r, \varepsilon)$ 
4:     if  $\mathcal{A}.\text{HASRESULT}(\mathcal{R}_{j,r})$  then
5:       load cached metrics
6:     else
7:       run simulator and store trace hash + metrics
      in  $\mathcal{A}$ 
8:     end if
9:   end for
10: end for
11: compute aggregates and confidence intervals

```

wincan expose optimization hooks that treat the simulator as a stochastic objective oracle:

$$\min_{\theta \in \Theta} \mathbb{E}_{\xi} [g(\mathcal{T}_{\xi, \varepsilon}(\theta))] \quad \text{s.t.} \quad \text{constraints in GaussIR.}$$

For continuous parameters, Bayesian optimization is a practical option [45]; for constrained policy learning, safe RL methods can be integrated (Section IX).

D. UNCERTAINTY QUANTIFICATION

When scenarios include parameter distributions, GaussTwin can run Monte Carlo sweeps and report:

- point estimates (mean/median),
- uncertainty intervals (e.g., 5–95% quantiles),
- risk measures (CVaR, probability of constraint violation).

These statistics align with the platform’s emphasis on decision support rather than single deterministic forecasts.

XII. VISUALIZATION, ANALYTICS, AND REPORTING

A digital twin is only useful if stakeholders can interrogate it: explore scenarios, identify bottlenecks, and understand why the model predicts certain outcomes. GaussTwintherefore treats visualization and reporting as part of the core architecture rather than an afterthought.

A. VISUALIZATION MODALITIES

GaussTwin supports multiple modalities, each derived from the same deterministic trace and metadata.

B. TRACE-TO-UI DATA PIPELINE

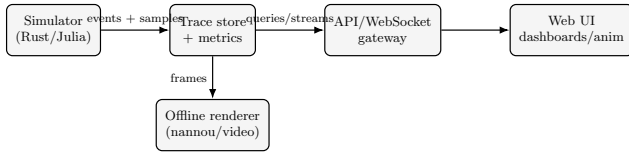
Figure 6 sketches a trace-to-UI pipeline that supports both interactive streaming updates and deterministic playback.

C. DETERMINISTIC ANIMATION RENDERING

For ABM models, visual debugging is often best achieved through deterministic animation that can be replayed exactly for a given run key $\mathcal{R}$. GaussTwin can provide a standard rendering API that maps agent state to visual primitives

TABLE 11. Visualization and Reporting Modalities

Modality	Typical use
Real-time dashboards	Monitoring live KPIs, alarms, and uncertainty intervals during streaming sync.
Scenario comparison	Side-by-side plots and tables across scenarios/seeds; risk and sensitivity analysis.
Agent animation	Spatial or network animations for ABM components (e.g., AGV movement, crowd flow).
Causal/structural views	Graph views of entities and couplings; SD stock-flow diagrams; event dependency graphs.
Report export	PDF/HTML reports and reproducible experiment bundles with hashes and configs.

**FIGURE 6.** Trace-to-UI pipeline: deterministic traces feed interactive dashboards and optional offline video rendering.

(positions, colors, labels) and writes frames or streaming updates.

Remark 4. Deterministic rendering is an important complement to Theorem 2: when simulation traces are reproducible, visual artifacts (videos, GIFs, interactive replays) also become reproducible and shareable for debugging and stakeholder communication.

XIII. INDUSTRY USE CASES AND COMPETITIVE LANDSCAPE

Digital twins span a wide range of industries, but successful deployments share a common pattern: a domain data model (assets and relationships), a set of operational questions (KPIs and constraints), and a coupling of continuous dynamics with discrete workflows and agent decision-making. This section provides background on representative industries and positions GaussTwin against widely used commercial platforms.

A. INDUSTRY BACKGROUNDS AND REPRESENTATIVE USE CASES

Table 12 summarizes typical use cases, required modeling paradigms, data sources, and decision metrics.

a: Why multi-paradigm matters.

In practice, stakeholders often need answers that cut across paradigms: e.g., a manufacturing twin must connect DES queues (jobs, machines) to SD energy/wear dynamics, while ABM captures heterogeneous operator/vehicle behavior and MARL enables policy learning for dispatch/routing. Platforms that provide only a graph/ontology layer typically require external simulators for these dynamics; pure simulation

Algorithm 8 Deterministic Frame Rendering (Conceptual)

Require: run key $\mathcal{R}$, trace $\mathcal{T}$, render map ψ , frame rate F

- 1: init renderer with deterministic config derived from $\mathcal{R}$
- 2: **for** $n = 0$ to $\lfloor F \cdot T \rfloor$ **do**
- 3: $t_n \leftarrow n/F$; interpolate state $z(t_n)$ from trace
- 4: scene $\leftarrow \psi(z(t_n))$ ▷ map state to draw commands
- 5: render frame n (stream or store)
- 6: **end for**

suites often lack a governed, enterprise-grade asset graph and streaming synchronization.

B. DETAILED INDUSTRY DEEP-DIVES

1) Manufacturing: Smart Factory Digital Twins

Modern manufacturing facilities require real-time monitoring and optimization across multiple scales:

- **Production planning horizon:** weeks to months; typically SD for aggregate capacity and inventory.
- **Operational scheduling:** hours to days; DES for job shop scheduling, resource allocation, and bottleneck analysis.
- **Real-time control:** seconds to minutes; ABM for individual machine/AGV agents, with MARL for dispatch policies.
- **Physics-informed models:** wear, thermal dynamics, energy consumption modeled via SD ODEs coupled to DES utilization.

a: Industry pain points.

- 1) **Data silos:** MES, SCADA, ERP systems use different schemas and update frequencies (1–1000 Hz).
- 2) **Model fragmentation:** separate tools for capacity planning (spreadsheets/LP solvers), detailed simulation (AnyLogic, Arena), and control (PLC logic).
- 3) **Lack of digital thread:** no unified versioning linking CAD/PLM models to operational simulation.

b: Quantitative impact.

Industry reports cite 10–20% improvements in OEE, 5–15% energy savings, and 20–30% reduction in unplanned downtime when integrated digital twins are deployed with real-time synchronization and predictive maintenance.

2) Supply Chain and Logistics: Network Resilience

Supply chains are inherently multi-echelon, multi-modal networks with stochastic demand, transit times, and disruptions.

- **Strategic network design:** SD for aggregate flows and inventory policies.
- **Tactical planning:** DES for warehouse operations, order fulfillment, and transportation modes.
- **Operational execution:** ABM for heterogeneous vehicle/driver agents; MARL for routing under real-time traffic and demand.

TABLE 12. Representative Digital Twin Use Cases and Modeling Needs

Industry	Typical twin questions	Paradigms needed	Data sources / KPIs
Manufacturing	Throughput, WIP, downtime impact, energy and wear, dispatch optimization	DES+ABM+SD (+MARL)	PLC/SCADA, MES, MTConnect/OPC UA; OEE, cycle time, energy, scrap
Supply chain & logistics	Network resilience, inventory policies, routing, congestion and delays	DES+ABM (+MARL)	ERP/WMS/TMS; service levels, lead time, cost-to-serve, emissions
Energy & utilities	Demand/asset forecasting, outage response, DER coordination, maintenance planning	SD+DES (+MARL)	AMI/SCADA; reliability indices, losses, cost, carbon intensity
Buildings & campuses	HVAC optimization, occupancy-driven control, comfort vs cost	SD+DES	BMS/IoT; comfort, energy, peak demand
Smart mobility & cities	Incident response, signal control, fleet allocation, crowd/evacuation	ABM+SD+DES (+MARL)	Sensors, GTFS, V2X; delay, safety, throughput
Healthcare operations	Patient flow, staffing, bed capacity, infection spread policies	DES+ABM	EHR/ADT feeds; wait times, LOS, utilization, safety events
Telecom & networks	Capacity planning, anomaly response, SLA management	DES (+ABM)	Telemetry; latency, drop rate, SLA violations

a: Business drivers.

- 1) **Resilience analysis:** simulate disruptions (port closures, supplier failures) and quantify inventory/cost trade-offs.
- 2) **Sustainability reporting:** integrate emissions models (SD for fuel consumption) with route optimization (MARL).
- 3) **Customer experience:** predict and optimize service levels (on-time delivery, lead time variability).

b: Typical model scale.

A regional logistics network may involve 50–500 facilities, 1000–10000 SKUs, and 100–1000 vehicles, yielding DES models with 10^5 – 10^7 events per simulated week.

3) Energy and Utilities: Grid Modernization and DER Integration

Electric grids are transitioning from centralized generation to distributed energy resources (DER): solar, wind, storage, and demand response.

- **Long-term planning:** SD for capacity expansion, renewable penetration, and policy scenarios (carbon pricing).
- **Operational reliability:** DES for outage response, maintenance scheduling, and crew dispatch.
- **Real-time coordination:** ABM for prosumer agents; MARL for DER aggregation and voltage regulation.

a: Regulatory and technical challenges.

Grid operators must balance reliability (SAIDI/SAIFI metrics), cost, and decarbonization targets under uncertainty (weather, demand, market prices). Digital twins enable scenario analysis and coordination algorithms that are validated offline before deployment.

b: Data integration.

Advanced Metering Infrastructure (AMI) provides 15-min to hourly consumption data for millions of meters; SCADA provides sub-second updates for substations and feeders. Synchronizing these heterogeneous streams with physics-based grid models is a canonical streaming twin use case.

4) Smart Mobility and Cities: Traffic Management and Fleet Optimization

Urban mobility systems combine infrastructure (roads, signals, transit), vehicle fleets (buses, shared mobility, emergency services), and human behavior (commuters, pedestrians).

- **Strategic planning:** SD for mode share, congestion pricing, and long-term infrastructure investment.
- **Tactical operations:** DES for transit schedules, incident management, and special event planning.
- **Real-time control:** ABM for individual vehicle/pedestrian agents; MARL for adaptive signal control and dynamic routing.

a: Performance metrics.

Cities measure mobility system performance via average delay, throughput (vehicles/hour), emissions, safety (crashes/year), and equity (accessibility across income groups). Multi-objective optimization under uncertainty is a natural fit for digital twin scenario sweeps.

b: Simulation scale and fidelity.

A city-scale traffic model may track 10^4 – 10^6 vehicles over a simulation day, requiring efficient ABM engines and spatial indexing. Coupling microscopic traffic dynamics (ABM) with macroscopic density models (SD) via boundary conditions is a hybrid modeling challenge addressed by GaussTwin's unified semantics.

5) Healthcare Operations: Patient Flow and Capacity Management

Hospitals and healthcare systems face capacity constraints, demand variability (seasonal, pandemic surges), and regulatory requirements (wait time targets, staff-to-patient ratios).

- **Capacity planning:** SD for aggregate patient volumes, bed occupancy, and staffing levels.
- **Flow optimization:** DES for patient pathways (ED, surgery, ICU, discharge), resource contention, and scheduling.

- **Epidemic modeling:** ABM for contact networks and infection spread; SD for population-level SIR/SEIR dynamics.

a: Operational questions.

- 1) How many ICU beds are needed to maintain <5% rejection probability under peak demand?
- 2) What staffing policies minimize overtime cost while meeting care standards?
- 3) How do triage protocols and isolation policies affect infection spread and LOS?

b: Data sources and privacy.

Electronic Health Records (EHR) and Admission/Discharge/Transfer (ADT) systems provide timestamped patient events, but privacy regulations (HIPAA, GDPR) require de-identification and governance controls—motivating GaussTwin’s emphasis on artifact versioning and audit logging.

6) Telecommunications and Networks: SLA Management and Anomaly Response

Telecom networks serve millions of customers with strict latency/throughput SLAs and must dynamically reallocate capacity under failures and demand spikes.

- **Network dimensioning:** SD for aggregate traffic growth and capacity expansion.
- **Failure analysis:** DES for link/node failures, rerouting, and restoration protocols.
- **Demand modeling:** optional ABM for user behavior (e.g., flash crowds, IoT device activation).

a: Twin objectives.

Predict SLA violations under what-if scenarios (regional outage, software bug, DDoS); quantify risk and cost of redundancy/backup capacity; optimize routing and load balancing policies.

b: Streaming telemetry.

Modern networks emit rich telemetry (SNMP, NetFlow, syslog) at rates of 10^3 – 10^6 events/sec across distributed nodes. A real-time digital twin ingests aggregated metrics and updates state estimates to trigger alarms and mitigation actions.

C. COMPETITORS AND POSITIONING

The digital twin market is fragmented across (i) simulation-first vendors, (ii) cloud graph/ontology twin platforms, and (iii) operations/decision platforms that integrate data, modeling, and governance. Representative competitors include Ansys (Twin Builder / ModelCenter), AnyLogic, Microsoft Azure Digital Twins, AWS IoT TwinMaker, Siemens Simcenter, Dassault 3DEXPERIENCE, PTC ThingWorx, and Palantir (Foundry/AIP).

1) Ansys Twin Builder and ModelCenter

a: Strengths.

Ansys Twin Builder is purpose-built for engineering digital twins with emphasis on physics-based ROM (Reduced Order Models) and FMI/Modelica co-simulation. It integrates with Ansys multiphysics solvers (structural, thermal, electromagnetic) and supports system-level simulation with VHDL-AMS and Simplorer. ModelCenter orchestrates workflows across heterogeneous engineering tools (CAD, FEA, CFD, optimization) with process integration and design exploration (PIDO) capabilities.

b: Limitations for operational twins.

- **Paradigm focus:** strong on continuous physics; limited native support for DES/ABM workflows common in operations research.
- **Data integration:** designed for engineering files (CAD, FEA meshes) rather than enterprise transactional data (ERP, MES, IoT streams).
- **AI/ML integration:** primarily via external Python scripts; no native MARL training loops or LLM-assisted compilation.
- **Deployment:** desktop/HPC-oriented; web deployment and real-time streaming synchronization require custom integration.

c: Target users.

Mechanical, electrical, and systems engineers performing design validation, predictive maintenance for complex assets (turbines, aircraft), and virtual prototyping.

2) AnyLogic

a: Strengths.

AnyLogic is a leading multimethod simulation platform explicitly supporting SD, DES, and ABM within a unified modeling environment. It provides graphical modeling (drag-and-drop blocks), Java extensibility, and libraries for supply chain, healthcare, pedestrian dynamics, and manufacturing. AnyLogic Cloud enables web deployment and scenario management; optimization experiments and parameter sweeps are built-in.

b: Limitations relative to GaussTwin.

- **Semantic formality:** multimethod coupling is pragmatic but lacks a published unified mathematical semantics or reproducibility guarantees (no formal trace hashing or deterministic orchestration contract).
- **Agentic compilation:** no LLM-assisted pipeline; users author models manually via GUI or Java code.
- **Governance and versioning:** project versioning exists but artifact-level immutability, content-addressable storage, and certification workflows are not first-class platform features.
- **Backend performance:** Java runtime; no native Rust backend or GPU-accelerated ML tight integration (MARL typically requires external Python/TensorFlow coupling).

c: Target users.

Operations researchers, supply chain analysts, consultants performing scenario analysis and business process optimization; strong in industries with established DES/SD/ABM modeling cultures.

d: Market position.

AnyLogic is widely adopted in academia and consulting; reported use by Fortune 500 companies for logistics, manufacturing, and healthcare operations. Pricing is proprietary (per-seat licenses + cloud runtime fees).

3) Microsoft Azure Digital Twins

a: Strengths.

Azure Digital Twins (ADT) provides a cloud-native twin graph platform built on DTDL (Digital Twins Definition Language), an ontology schema language. It integrates tightly with Azure IoT Hub, Azure Data Explorer, and Power BI for data ingestion, querying, and visualization. ADT emphasizes graph-based representations and real-time data synchronization for IoT scenarios (smart buildings, industrial IoT).

b: Architectural philosophy.

ADT positions the twin as a knowledge graph that mirrors the physical asset topology and state, with simulation/control as external services. This separation enables integration of heterogeneous simulation tools (AnyLogic, MATLAB, custom models) via adapters, but places the burden of orchestration, semantics, and reproducibility on the user.

c: Limitations for autonomous multi-paradigm twins.

- **No native simulation:** ADT does not embed DES/SD/ABM engines; users must implement simulation externally and synchronize via APIs.
- **No unified semantics:** DTDL defines schema and relationships but does not specify hybrid transition semantics, event ordering, or continuous-discrete coupling rules.
- **Reproducibility:** no built-in trace hashing, deterministic RNG, or versioned artifact contracts; reproducibility depends on external simulation implementations.
- **MARL/AI:** Azure Machine Learning and Azure OpenAI can be integrated, but there is no certified agentic compilation pipeline or MARL-in-the-loop design with safety guarantees.

d: Target users.

Enterprise IT and cloud architects integrating IoT data streams with business applications; strong fit for graph-centric use cases (facility management, asset tracking) where simulation is secondary.

e: Pricing and scale.

Pay-as-you-go based on API calls, query units, and data ingress; scales to millions of twins and real-time telemetry streams; operational cost depends on query/update frequency.

4) AWS IoT TwinMaker

a: Strengths.

AWS IoT TwinMaker provides a managed service for creating digital twins of physical systems by connecting IoT data (AWS IoT SiteWise, Timestream), 3D models (from CAD tools), and knowledge graphs. It emphasizes visual composition and dashboards with integrated 3D scene rendering (via Babylon.js/ThreeJS).

b: Architectural approach.

Similar to Azure DT, TwinMaker positions the twin as a data integration and visualization layer; simulation/analytics are implemented via AWS Lambda, SageMaker, or external tools.

c: Limitations.

- **No embedded multi-paradigm simulation:** users must build DES/SD/ABM models externally.
- **No formal semantics or certification:** relies on user-defined Lambda functions and external models; no typed IR, deterministic orchestration, or LLM guardrails.
- **Vendor lock-in:** tightly coupled to AWS services; portability to other clouds or on-prem requires re-architecting.

d: Target users.

AWS customers with existing IoT/SiteWise deployments seeking operational dashboards and 3D visualization; less focused on analytical simulation and optimization.

5) Palantir Foundry and AIP (Artificial Intelligence Platform)

a: Strengths.

Palantir Foundry is an enterprise data integration and analytics platform with strong governance, lineage tracking, and access control. AIP (released 2023) integrates LLMs for data wrangling, natural language queries, and agentic workflows. Foundry emphasizes data ontology (entities, relationships, properties) and provides code workspaces (Python, SQL, Spark) for custom analytics and modeling.

b: Modeling and simulation capabilities.

Foundry does not provide native DES/SD/ABM engines; users implement simulation models in Python/Julia within Foundry workspaces or integrate external tools (AnyLogic, MATLAB) via pipelines. Foundry's value proposition is governed data integration, not simulation semantics.

c: AIP and LLM integration.

AIP enables LLM-assisted data exploration, report generation, and conversational interfaces. However, as of late 2024, AIP does not provide a certified agentic compilation pipeline with formal validators and typed IR for simulation code generation. LLM outputs are treated as productivity aids rather than verified artifacts.

d: Differentiation from GaussTwin.

- **Governance and data lineage:** Foundry excels at tracking data provenance and access policies across complex enterprise data estates; GaussTwin focuses on simulation artifact governance (snapshots, IR, scenarios).
- **Simulation semantics:** Foundry is agnostic; GaussTwin provides a unified hybrid semantics and reproducibility guarantees.
- **Target problems:** Foundry targets data integration, BI, and ML pipelines for operational intelligence; GaussTwin targets scenario analysis, what-if optimization, and real-time synchronization for decision support.

e: Target users.

Defense, intelligence, and large enterprises requiring auditability and data governance; Palantir has expanded into commercial sectors (manufacturing, logistics, healthcare) with AIP as a differentiation layer.

6) Siemens Simcenter, Dassault 3DEXPERIENCE, PTC ThingWorx

These platforms represent the PLM (Product Lifecycle Management) and industrial IoT vendor ecosystem:

- **Siemens Simcenter:** simulation suite (1D/3D CFD, FEA, system simulation) integrated with Teamcenter PLM and MindSphere IoT; strong on engineering analysis and virtual commissioning.
- **Dassault 3DEXPERIENCE:** CAD/PLM platform (CATIA, SIMULIA, DELMIA) with virtual twin capabilities for design, manufacturing, and operations; emphasizes collaboration and 3D visualization.
- **PTC ThingWorx:** IoT platform with augmented reality (Vuforia), CAD (Creo), and PLM (Windchill); digital twin capabilities via ThingWorx Navigate and ThingWorx Studio.

a: Common pattern.

These vendors provide federated ecosystems of tools with proprietary connectors; digital twin capabilities are achieved through integration rather than a unified semantic layer. Strengths lie in CAD/CAM/PLM workflows and industrial asset management; weaknesses relative to GaussTwin include limited formal semantics for multi-paradigm simulation, no certified agentic compilation, and vendor lock-in.

D. FEATURE COMPARISON (HIGH-LEVEL)

Table 13 provides a qualitative comparison emphasizing GaussTwin's intended differentiation: a typed IR and certified compilation pipeline that reduces semantic drift and increases auditability for AI-assisted workflows.

Remark 5 (Reading the table correctly). This table is intentionally qualitative: vendors evolve quickly and feature scope depends on licensing and integrations. The key technical claim for GAUSSERTWIN is not "more features," but a *stronger semantic contract*: certified IR + deterministic orchestration

enabling reproducible, cross-backend, AI-assisted twin construction.

E. DETAILED CAPABILITY MATRIX COMPARISON

Table 14 provides a more granular comparison across key technical dimensions.

F. MARKET POSITIONING AND STRATEGIC DIFFERENTIATION

GaussTwin targets the intersection of three underserved needs:

- 1) **Multi-paradigm rigor:** users who require SD, DES, ABM, MARL but find existing multimethod tools lack formal semantics, reproducibility, and cross-platform portability.
- 2) **AI-assisted productivity with safety:** organizations adopting LLMs for modeling/analytics but lacking governance frameworks to certify generated artifacts.
- 3) **Open and interoperable:** teams seeking vendor-neutral, standards-compatible twins that can integrate with existing PLM/ERP/MES systems and future FMI/HLA federations.

a: Intended user segments.

- Industrial R&D teams blending engineering physics (Ansys-like) with operational workflows (AnyLogic-like).
- Operations researchers and data scientists requiring reproducible scenario analysis with ML/MARL integration.
- Regulated industries (aerospace, healthcare, energy) where auditability, certification, and deterministic replay are compliance requirements.
- Academic/research groups seeking an open platform for hybrid simulation methods development.

XIV. EVALUATION METHODOLOGY AND CASE STUDIES

A. METRICS

We recommend evaluating:

- **Expressivity coverage:** SD/DES/ABM/MARL components and coupling patterns (Table 5).
- **Runtime performance:** events/sec, ODE throughput, memory, latency under streaming updates.
- **Reproducibility:** trace hash equality across repeated runs; sensitivity to parallelism.
- **Agentic pipeline reliability:** validator pass rates; defect types detected.
- **MARL performance:** reward, constraint violations, robustness under disturbances.

B. BENCHMARK SUITE DESIGN

To support comparable and reproducible evaluation across implementations, we recommend a benchmark suite containing:

- **Microbenchmarks:** event queue throughput (push/pop/reschedule), agent update kernels, ODE integrator throughput, and serialization costs for trace streaming.

TABLE 13. High-Level Competitor Comparison (Qualitative)

Platform	Multi-method simulation	Twin graph/ontology	AI/agents integration	Governance focus	Notes
GAUSSTWIN(proposed)	✓(SD/DES/ABM)	optional adapters)	(via ✓(certified agentic compilation; MARL hooks)	✓	Typed GAUSSIR+ deterministic validators + Rust/Julia backends
AnyLogic [14], [15]	✓(SD/DES/ABM)	limited	limited	moderate	Strong multimethod modeling environment; enterprise integration varies
Ansys Twin Builder [16]	focused (physics/FMUs)	limited	limited	moderate	Strong engineering/physics workflows; co-sim via FMI
Ansys ModelCenter [17]	orchestration	n/a	limited	moderate	Workflow/process integration for engineering models
Azure Digital Twins [11]–[13]	external	✓	external	✓	DTDL + graph; simulation/control typically external services
AWS IoT TwinMaker [18]	external	✓	external	✓	Integrates IoT + 3D scenes; simulation typically external
Palantir Foundry/AIP [19], [20]	external	data graph	✓	✓	Strong data integration/governance; modeling via integrated tools/partners

TABLE 14. Detailed Competitor Capability Matrix

Capability	GAUSSTWIN	AnyLogic	Ansys Twin Builder	Azure DT	Palantir	FMI/HLA	Notes
Unified SD/DES/ABM	✓	✓	partial	n/a	n/a	n/a	AnyLogic multimethod, Ansys physics-focused
Formal hybrid semantics	✓	informal	informal	n/a	n/a	FMI co-sim	GAUSSTWIN: GHTP + proofs
Deterministic replay	✓	partial	partial	user impl.	user impl.	partial	GAUSSTWIN: trace hashing + RNG contracts
LLM-assisted authoring	✓	no	no	no	partial (AIP)	n/a	GAUSSTWIN: certified pipeline
Typed IR / schema	✓(GAUSSIR)	proprietary	proprietary	DTDL	ontology	FMI XML	GAUSSTWIN: open, versioned IR
Cross-backend portability	✓(Rust/Julia)	Java only	solver-dependent	cloud only	Python/Spark	✓	FMI enables tool coupling
Real-time sync	✓	limited	limited	✓	✓	adapter	Azure/Palantir strong on IoT ingestion
MARL training loops	✓	external	external	external	external	n/a	GAUSSTWIN: native Candle/tch-rs
Governance/audit	✓	moderate	moderate	✓	✓	user impl.	Palantir/Azure: enterprise data gov
Open-source	roadmap	no	no	no	no	✓	FMI/HLA are open standards

- **Coupling benchmarks:** canonical SD–DES and ABM–DES coupling patterns (Table 5) with known qualitative behavior (e.g., oscillatory feedback, congestion collapse).
- **Streaming benchmarks:** synthetic observation streams with controlled latency and noise to test synchronization stability (Algorithm 5).
- **MARL benchmarks:** decision points embedded in DES/ABM loops with clear constraints and evaluation metrics (Algorithm 6).

For each benchmark, the suite should define a canonical

snapshot, scenario set, and seed list, enabling cross-backend trace hash comparisons.

C. EVALUATION OF THE AGENTIC COMPILATION PIPELINE

Beyond simulation runtime, GaussTwin must be evaluated on compilation reliability:

- validator pass rate after each refinement iteration (Algorithm 1),
- distribution of defect categories (schema, type, unit, con-

- straint, static analysis),
- time-to-certificate and LLM token/cost budget,
- human review effort reduction relative to manual modeling.

A practical evaluation uses a corpus of real or synthetic spreadsheet snapshots with injected defects and measures how quickly the pipeline recovers certified artifacts.

D. ABLATIONS AND SENSITIVITY

Recommended ablations include:

- disabling LLM assistance (manual IR authoring) vs certified agentic compilation,
- template-restricted generation vs free-form generation (where allowed),
- Rust vs Julia backend for the same GaussIR (Figure 3),
- varying numerical tolerances and event tie-breaking strategies.

E. CASE STUDY 1: SMART MANUFACTURING CELL

A manufacturing cell with machines, buffers, and AGVs:

- DES: job arrivals, service events, failures/repairs.
- ABM: machine, AGV, and operator agents.
- SD: energy accumulation, wear dynamics.
- MARL: learn dispatch/routing policies for AGVs.

Example SD energy model:

$$\dot{E}(t) = \sum_{m=1}^M (p_m^{\text{idle}} + (p_m^{\text{act}} - p_m^{\text{idle}})u_m(t)), \quad (5)$$

where utilization $u_m(t)$ is derived from DES event logs.

F. CASE STUDY 2: URBAN MOBILITY

Traffic with vehicle agents (ABM), macroscopic density (SD), disruptions (DES), and signal control via MARL. Streaming sensors update densities and incidents.

G. COMPARATIVE ANALYSIS

XV. DISCUSSION AND LIMITATIONS

This paper focuses on a platform design that is simultaneously multi-paradigm, web-deployable, and compatible with LLM-assisted workflows. Several trade-offs are worth emphasizing.

A. DETERMINISM VS. THROUGHPUT

Deterministic event ordering (Assumption 2) is central for auditability and debugging, but it can constrain parallel execution. GaussTwin addresses this by:

- parallelizing read-only computations (e.g., agent perception, feature extraction),
- serializing state updates through a deterministic orchestrator (Algorithm 4),
- using batch processing and cache-aware sweeps (Algorithm 7) for throughput at the experiment level.

B. SINGLE SEMANTICS VS. STANDARDS-BASED COUPLING

A unified semantics simplifies reasoning and compilation, but ecosystems already depend on standards like FMI/HLA. GaussTwin therefore treats the unified semantics as an internal contract (via GaussIR), while planning adapters for standards-based federation when external interoperability is required.

C. LLM ASSISTANCE: PRODUCTIVITY VS. RISK

LLMs can substantially reduce modeling friction, especially for data cleaning and schema inference, but they introduce risks:

- hallucinated identifiers or constraints,
- incorrect units or sign errors in equations,
- injection attacks via uploaded content,
- uncontrolled code generation.

The certified pipeline (Algorithm 1) and template-restricted code generation (Section VI-H) mitigate these risks, but do not eliminate them. Human review remains important for high-stakes deployments.

D. MODELING LIMITATIONS

- **Stiff dynamics:** SD components may require stiff ODE solvers; coupling stiff ODEs with frequent events can be numerically challenging.
- **Calibration and identifiability:** Parameter estimation from partial observations can be ill-posed without additional priors or experiments.
- **Policy validity under distribution shift:** MARL policies trained in simulation may fail under real-world changes; robust evaluation and monitoring are required.
- **Scale of agent models:** Extremely large ABMs may demand specialized data structures and hardware acceleration beyond the baseline design.

E. RESEARCH OPPORTUNITIES

The hybrid semantics suggests several research directions:

- 1) proof-carrying code generation for larger GaussIR subsets,
- 2) automated invariant discovery to strengthen validator coverage,
- 3) co-simulation error bounds and adaptive stepping under mixed SD/DES dynamics,
- 4) safe and interpretable MARL for operational decision support.

XVI. SECURITY, GOVERNANCE, AND RESPONSIBLE DEPLOYMENT

A. THREAT MODEL FOR LLM-INTEGRATED PIPELINES

LLM-based workflows introduce risks including prompt injection via uploaded content, data exfiltration, and generation of unsafe code. GaussTwin mitigates these risks by:

- separating content from instructions (structured extraction before prompting),

TABLE 15. Qualitative Comparison

Platform / Standard	DES	SD	ABM	MARL	Notes
GAUSTRWIN(proposed)	✓	✓	✓	✓	Typed IR, certified LLM pipeline, Rust+Julia backends
AnyLogic	✓	✓	✓	limited	Commercial multimethod tool [14]
Azure Digital Twins	graph	graph	graph	n/a	Twin graph + DTDL; simulation external [11], [12]
FMI 3.0	co-sim	co-sim	co-sim	n/a	Standard for model exchange/co-simulation [21]
IEEE HLA 1516	federation	federation	federation	n/a	Distributed simulation standard [22]
Agents.jl	n/a	via SciML	✓	external	Julia ABM framework [2], [4]
krABMaga	✓	external	✓	external	Rust ABM engine [1]

- allowing only a fixed set of tools/actions for agentic steps,
- gating all LLM outputs behind deterministic validators and sandboxed execution,
- versioning and audit logging of all artifacts and prompts (where policy permits).

B. GOVERNANCE CONTROLS

- role-based access control for projects and data,
- configurable LLM policies (model allowlists; cloud vs on-prem),
- compliance-friendly artifact retention policies.

XVII. CONCLUSION

We proposed GaussTwin, a multi-paradigm digital twin platform that unifies SD, DES, ABM, and MARL through a generalized hybrid transition semantics and a certified agentic compilation workflow. GaussTwin bridges a spreadsheet-first user workflow with typed intermediate representations and reproducible Rust/Julia execution backends, while safely leveraging LLMs through deterministic validation and artifact versioning. We provided mathematical foundations, embedding theorems, and orchestration correctness results that guide the platform design, along with algorithms for compilation, hybrid simulation, streaming synchronization, and MARL training. Future work includes standards adapters (FMI/HLA/DTDL), deeper formal verification of GaussIR subsets and generated code, and large-scale benchmarks across industrial domains.

APPENDIX A APPENDIX A: EXAMPLE GAUSSIRSNIPPET (ILLUSTRATIVE)

```
entity Machine {
  id: String
  state: {Idle, Busy, Down}
  p_idle: Power[W]
  p_active: Power[W]
}
flow EnergyAccumulation {
  dE/dt = sum_m (m.p_idle + (m.p_active - m.p_idle) * util(m))
}
event JobArrival {
  intensity: Poisson(lambda=arrival_rate)
  reset: enqueue(job_queue, new_job())
}
agent AGV {
  state: {Location, Battery}
```

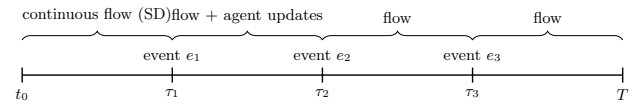


FIGURE 7. Hybrid execution timeline: continuous flows interleaved with discrete events.

```
action: route_to(machine_id)
reward: -wip - energy - tardiness
}
```

APPENDIX B APPENDIX B: HYBRID TIMELINE VISUALIZATION

APPENDIX C APPENDIX C: GAUSSIRGRAMMAR (SKETCH)

The following is an illustrative BNF-style sketch of a minimal GaussIR grammar. Production rules are intentionally simplified for readability; the platform implementation should be versioned and machine-readable (e.g., JSON Schema + a DSL parser).

```
<program> ::= <entities> <state> <flows> <events> <agents>
<entities> ::= ("entity" <id> "{" <field>* "}")*
<field> ::= <id> ":" <type>
<state> ::= "state" "{" ("continuous" <idlist> ";")?
                    ("discrete" <id> ":" <enum> ";")*
<flows> ::= ("flow" <id> "{" <odeeq>* "}")*
<odeeq> ::= "d" <id> "/dt" "=" <expr>
<events> ::= ("event" <id> "{" <hazard>? <guard>? <reset> "}")*
<hazard> ::= "intensity" ":" <expr>
<guard> ::= "guard" ":" <expr>
<reset> ::= "reset" ":" <stmtlist>
<agents> ::= ("agent" <id> "{" <field>* <update>* "}")*
<update> ::= ("on_event" <id> ":" <stmtlist>) | ("tick" ":" <stmtlist>)
<rl> ::= "rl" "{" ("obs" <id> ":" <expr>)* ("action" <id> ":" <stmtlist>)*
<scenarios> ::= ("scenario" <id> "{" <override>* <intervent>*)
```

APPENDIX D APPENDIX D: REPRODUCIBILITY CHECKLIST

Table 16 provides a practical reproducibility checklist aligned with Theorem 2 and with the platform artifact tuple $\mathcal{R}$.

APPENDIX E APPENDIX E: EXCEL SNAPSHOT TEMPLATE (ILLUSTRATIVE)

A spreadsheet-first workflow benefits from a stable template. Table 17 lists illustrative sheets and their responsibilities.

TABLE 16. Reproducibility Checklist for GAUSSTWINExperiments

Item	Requirement
Snapshot versioning	Persist a content hash of all Excel inputs, including schema version.
Scenario versioning	Persist a content hash of scenario DSL/GAUSSIRoverlays.
IR and code hashes	Persist GAUSSIRhash and generated backend code hash; store compiler version.
RNG seeds	Store per-run seeds; define deterministic RNG algorithm and seeding scheme.
Numerical tolerance	Store ODE solver settings, tolerances, and floating-point mode.
Deterministic ordering	Define total ordering for simultaneous events and for parallel computations.
Environment capture	Record OS, compiler versions, Rust/Julia package versions, and hardware info.
Trace hashing	Store a canonical trace hash and key summary metrics for regression testing.
Streaming replay	Log observation stream with timestamps/hashes and filter configuration.

TABLE 17. Illustrative Excel Sheets for Snapshot and Scenarios

Sheet	Purpose
Entities	IDs, types, and static attributes (capacity, cost, location).
Relations	Graph edges (connectivity, routes, dependencies).
Parameters	Global parameters with units and default values.
InitialState	Initial values for continuous/discrete variables and agent fields.
Scenarios	Scenario definitions: overrides, distributions, interventions.
Policies	Optional policy settings or MARL configuration.
Observations	Mapping from sensor channels to model variables (for sync).

APPENDIX F APPENDIX F: SECURITY NOTES FOR SANDBOX EXECUTION

To safely execute generated code, GaussTwin should sandbox compilation and runtime:

- run backend compilation in isolated containers with restricted network access,
- enforce a denylist/allowlist for system calls and file access,
- cap CPU/memory/time per run, and limit output size,
- separate secrets (API keys) from runtime environments to prevent exfiltration.

ACKNOWLEDGMENT

The authors thank TBD.

REFERENCES

- [1] krABMaga Organization, “krABMaga: A discrete events simulation engine for developing agent-based models in Rust,” GitHub repository. Accessed: Dec. 22, 2025. [Online]. Available: <https://github.com/krABMaga/krABMaga>
- [2] A. R. Vahdati, “Agents.jl: agent-based modeling framework in Julia,” *Journal of Open Source Software*, vol. 4, no. 42, p. 1611, 2019, doi: 10.21105/joss.01611.
- [3] G. Datseris, A. R. Vahdati, and T. C. DuBois, “Agents.jl: a performant and feature-full agent based modelling software of minimal code complexity,” *arXiv:2101.10072*, 2021. [Online]. Available: <https://arxiv.org/abs/2101.10072>
- [4] G. Datseris, A. R. Vahdati, and T. C. DuBois, “Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity,” *Simulation*, 2022, doi: 10.1177/00375497211068820.
- [5] Hugging Face, “Candle documentation.” Accessed: Dec. 22, 2025. [Online]. Available: <https://huggingface.github.io/candle/>
- [6] Hugging Face, “candle: Minimalist ML framework for Rust,” GitHub repository. Accessed: Dec. 22, 2025. [Online]. Available: <https://github.com/huggingface/candle>
- [7] L. Mazare, “tch-rs: Rust bindings for the C++ API of PyTorch (libtorch),” GitHub repository. Accessed: Dec. 22, 2025. [Online]. Available: <https://github.com/LaurentMazare/tch-rs>
- [8] OpenRouter, “OpenRouter documentation.” Accessed: Dec. 22, 2025. [Online]. Available: <https://openrouter.ai/docs>
- [9] OpenRouter, “OpenRouter API reference.” Accessed: Dec. 22, 2025. [Online]. Available: <https://openrouter.ai/docs/api/reference/overview>
- [10] nannou-org, “nannou: A creative coding framework for Rust,” GitHub repository. Accessed: Dec. 22, 2025. [Online]. Available: <https://github.com/nannou-org/nannou>
- [11] Microsoft, “Azure Digital Twins overview.” Accessed: Dec. 22, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/digital-twins/overview>
- [12] Microsoft, “DTDL models in Azure Digital Twins.” Accessed: Dec. 22, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/digital-twins/concepts-models>
- [13] Microsoft, “Digital twins and the twin graph - Azure Digital Twins.” Accessed: Dec. 22, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/digital-twins/concepts-twins-graph>
- [14] AnyLogic, “AnyLogic platform features.” Accessed: Dec. 22, 2025. [Online]. Available: <https://www.anylogic.com/features/>
- [15] AnyLogic Help, “Agent-based modeling and simulation (AnyLogic documentation).” Accessed: Dec. 22, 2025. [Online]. Available: <https://anylogic.help/anylogic/agentbased/index.html>
- [16] Ansys, “Ansys Twin Builder.” Accessed: Dec. 22, 2025. [Online]. Available: <https://www.ansys.com/products/digital-twin/ansys-twin-builder>
- [17] Ansys, “Ansys ModelCenter.” Accessed: Dec. 22, 2025. [Online]. Available: <https://www.ansys.com/products/system/ansys-modelcenter>
- [18] Amazon Web Services, “AWS IoT TwinMaker.” Accessed: Dec. 22, 2025. [Online]. Available: <https://aws.amazon.com/iot-twinmaker/>
- [19] Palantir, “Palantir Foundry.” Accessed: Dec. 22, 2025. [Online]. Available: <https://www.palantir.com/platforms/foundry/>
- [20] Palantir, “Artificial Intelligence Platform (AIP).” Accessed: Dec. 22, 2025. [Online]. Available: <https://www.palantir.com/platforms/aip/>
- [21] Modelica Association, “Functional Mock-up Interface (FMI) 3.0 specification.” Accessed: Dec. 22, 2025. [Online]. Available: <https://fmi-standard.org/docs/3.0/>
- [22] IEEE Standards Association, “IEEE 1516-2025: High Level Architecture (HLA) for distributed simulation.” Accessed: Dec. 22, 2025. [Online]. Available: <https://standards.ieee.org/ieee/1516/6687/>
- [23] I. David, D. Tilbury, C. Gomes, and G. Shao, “Interoperability of digital twins: challenges, success stories, and a roadmap,” *NIST/ISoLA report*, 2024. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=956427
- [24] ISO, “ISO 23247-1:2021 Digital twin framework for manufacturing.” 2021. [Online]. Available: <https://cdn.standards.iteh.ai/samples/75066/ec0a1c59176e488887873acda6b7ecd9/ISO-23247-1-2021.pdf>
- [25] OPC Foundation, “Interoperability for Industrie 4.0 and the Internet of Things (OPC UA).” 2023. [Online]. Available: <https://opcfoundation.org/wp-content/uploads/2023/05/OPC-UA-Interoperability-For-Industrie4-and-IoT-EN.pdf>
- [26] MTConnect Institute, “MTConnect standard.” Accessed: Dec. 22, 2025. [Online]. Available: <https://www.mtconnect.org/>
- [27] M. W. Grieves, “Digital Twin: Manufacturing excellence through virtual factory replication,” *White Paper*, 2014. [Online]. Available: <https://www.3ds.com/fileadmin/PRODUCTS-SERVICES/DELMIA/PDF/Whitepaper/DELMIA-APRISO-Digital-Twin-Whitepaper.pdf>
- [28] J. W. Forrester, *Industrial Dynamics*. MIT Press, 1961.

- [29] B. P. Zeigler, A. Muzy, and E. Kofman, *Theory of Modeling and Simulation: Discrete Event & Iterative System Computational Foundations*, 3rd ed. Academic Press, 2018.
- [30] H. Vangheluwe, "The Discrete Event System specification (DEVS) formalism," lecture notes, 2001. [Online]. Available: <https://www.cs.mcgill.ca/~hv/classes/MS/DEVS.pdf>
- [31] E. Bonabeau, "Agent-based modeling: Methods and techniques for simulating human systems," *Proc. Natl. Acad. Sci. USA*, vol. 99, no. suppl. 3, pp. 7280–7287, 2002.
- [32] J. S. Morgan, L. M. Howick, and A. B. Chick, "A toolkit of designs for mixing discrete event simulation and system dynamics," *European Journal of Operational Research*, vol. 257, no. 3, pp. 907–918, 2017.
- [33] A. Borshchev, "Multi-method modelling: AnyLogic," in *Discrete Event Simulation and Agent-Based Modelling*. Wiley, 2014.
- [34] S. Howick et al., "A framework for conceptualising hybrid system dynamics simulation," *European Journal of Operational Research*, 2024.
- [35] M. H. A. Davis, "Piecewise-deterministic Markov processes: A general class of non-diffusion stochastic models," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 46, no. 3, pp. 353–388, 1984.
- [36] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proc. ICML*, 1994.
- [37] D. Huh and P. Mohapatra, "Multi-agent reinforcement learning: a comprehensive survey," *arXiv:2312.10256*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.10256>
- [38] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," *arXiv:2210.03629*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [39] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," *arXiv:2303.17651*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.17651>
- [40] M. Chen et al., "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [41] OpenAI, "HumanEval: hand-written evaluation set." Accessed: Dec. 22, 2025. [Online]. Available: <https://github.com/openai/human-eval>
- [42] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME–Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [43] A. Doucet, N. de Freitas, and N. Gordon, Eds., *Sequential Monte Carlo Methods in Practice*. Springer, 2001.
- [44] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained Policy Optimization," in *Proc. International Conference on Machine Learning (ICML)*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.10528>
- [45] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.



RISMAN ADNAN received the B.Sc. and M.Sc. degrees in Theoretical Physics from Universitas Indonesia in 1998 and 2000, respectively, and the Ph.D. degree in Computer Science from the same institution in 2021. He served as Director of Developer Ecosystem at Microsoft Indonesia from 2004 to 2014 and as Chief Technology Officer at Samsung Research and Development Indonesia (SRIN) from 2014 to 2023. Since 2023, he has been Technology Director at Kalbe Digital Lab.

His research interests encompass bioinformatics, theoretical physics, machine learning, quantum computing, and healthcare technologies, including genomics, medical imaging, and AI-enabled real-time therapy (AIoRT) systems.

...